21.04.2026

Локальный поиск. Рандомизированные алгоритмы. Алгоритмы подсчета.

Филиппов Михаил Витальевич

m.filippov@g.nsu.ru

89232283872

Императивное программирование, 2024-2025

Новосибирский государственный университет

*НАСТОЯЩАЯ НАУКА

Давайте познакомимся

Филиппов Михаил Витальевич

- Окончил магистратуру ФФ НГУ
- Окончил аспирантуру ИТ СО РАН
- Являюсь м.н.с. ИТ СО РАН
- 8+ лет опыт в программировании C/C++
- к.ф.-м.н.

Новосибирский государственный университет
*НАСТОЯЩАЯ НАУКА

Адженда

Локальный поиск

45 минут

Рандомизированные алгоритмы

45 минут

Адженда

Локальный поиск

45 минут

Рандомизированные алгоритмы

45 минут

Введение

Локальный поиск — чрезвычайно универсальный прием; этим термином описываются любые алгоритмы, которые последовательно «исследуют» пространство возможных решений, перемещаясь за один шаг от текущего решения к другому, «ближнему».

Общность и гибкость этого метода имеют свои преимущества:

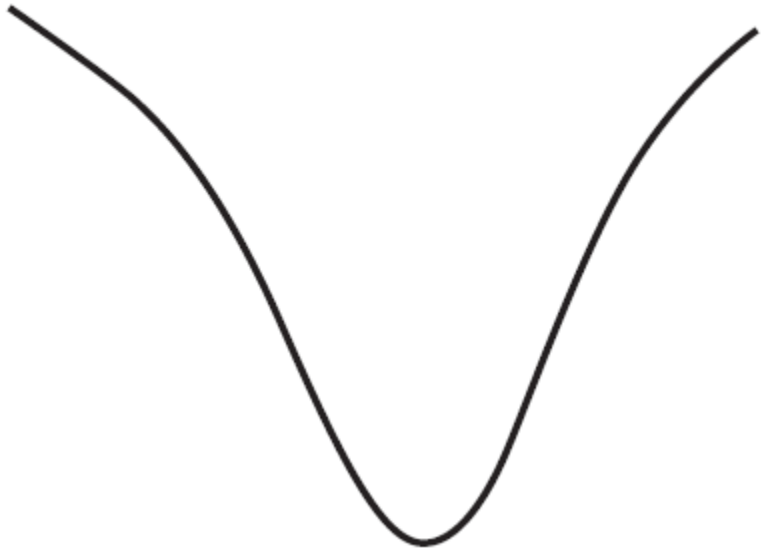
- алгоритм на базе локального поиска можно без особых трудностей разработать почти для любой вычислительно сложной задачи;
- часто бывает очень трудно сказать что-нибудь конкретное или доказуемое о качестве решений, находимых алгоритмом локального поиска, и, соответственно, очень сложно определить, хороший или плохой алгоритм локального поиска используется в каждом конкретном случае.

Алгоритмы локального поиска обычно представляют собой эвристики, предназначенные для нахождения хороших (но не обязательно оптимальных) решений вычислительных задач.

Полезная интуитивно понятная аналогия существует в принципе минимизации энергии в физике. Физические системы постоянно решают задачу минимизации, стремясь к минимуму потенциальной энергии. Чему можно научиться, наблюдая за минимизацией в природе? Не подскажет ли она какие-нибудь новые алгоритмы?

Потенциальная энергия

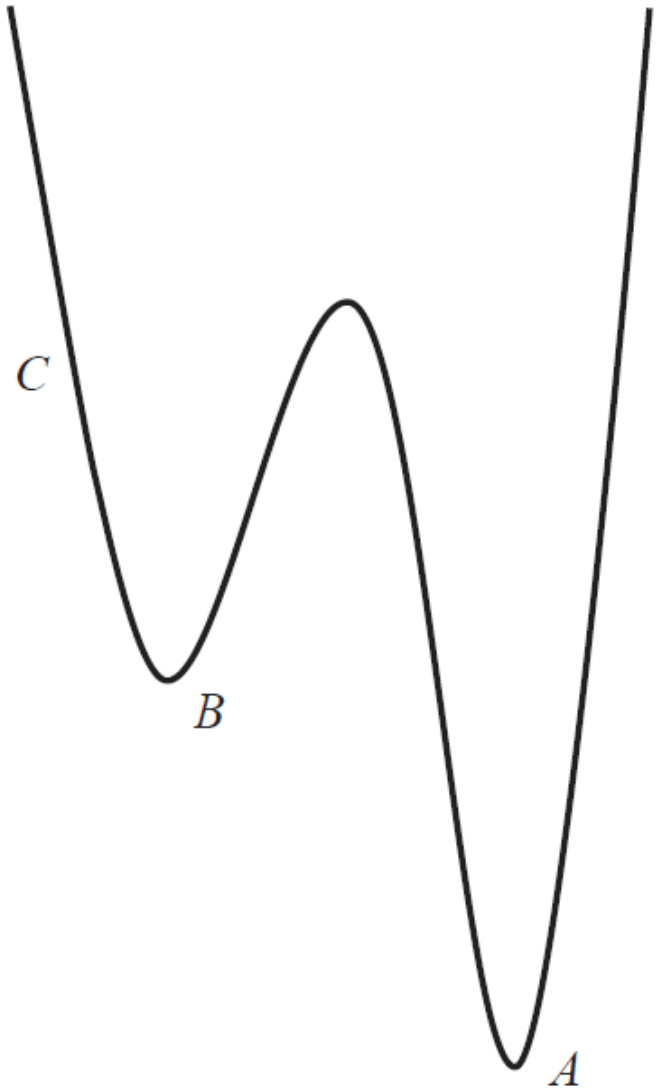
Если бы мир выглядел так, как нам рассказывают в начальном курсе механики, он состоял бы исключительно из хоккейных шайб, скользящих по льду, и шаров, скатывающихся по наклонным поверхностям. Шайба скользит, потому что ее толкнули; но почему шары скатываются вниз? Из ньютоновской механики известно, что шар стремится минимизировать свою потенциальную энергию. В частности, если шар имеет массу m и падает на расстояние h, он теряет потенциальную энергию, пропорциональную mh. Итак, если отпустить шар у верхнего края воронкообразного углубления, его потенциальная энергия будет минимальной в самой нижней точке.



Когда поверхность потенциальной энергии имеет структуру простой воронки, найти нижнюю точку достаточно легко

Потенциальная энергия

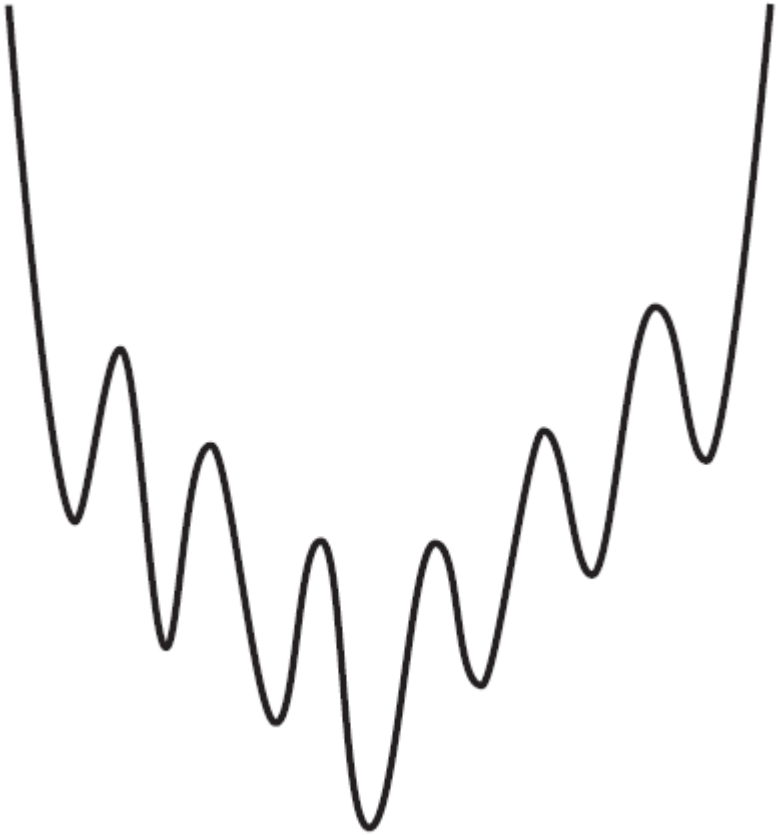
При усложнении структуры поверхности проявляются дополнительные сложности. Возьмем «двойную воронку». Точка А находится ниже точки В, поэтому она является предпочтительной для нахождения шара в состоянии покоя. Но если шар скатывается с точки С, он не сможет преодолеть барьер между двумя воронками и окажется в В. В таких случаях говорят, что шар попал в локальный минимум: он находится в самой нижней точке, если рассматривать окрестности текущего положения; но если отступить и взглянуть на ситуацию в целом, мы видим, что глобальный минимум не достигнут.



Большинство поверхностей сложнее простых воронок; например, в этой «двойной воронке» существуют большой глобальный минимум и меньший локальный минимум

Потенциальная энергия

Конечно, очень большие физические системы также должны стремиться к минимизации своей энергии. Представьте, например, что вы взяли несколько граммов однородного вещества, нагрели его и изучаете его поведение во времени. Чтобы точно отразить потенциальную энергию образца, теоретически необходимо представить поведение каждого атома вещества и его взаимодействие с ближними атомами. Но также будет полезно рассмотреть свойства системы в целом (то есть всего образца), и здесь на помощь приходит статистическая механика. Мы еще вернемся к статистической механике, а пока просто заметим, что концепция «поверхности потенциальной энергии» предоставляет полезную и наглядную аналогию для процесса, который используется в больших физических системах для минимизации энергии.



Обобщенная поверхность потенциальной энергии может содержать множество локальных минимумов, усложняющих поиск глобального минимума, как в изображенной на рисунке «гребенке»

Связь с оптимизацией

Подход к минимизации энергии базируется на нескольких основных положениях:

- физическая система может находиться в одном из многочисленных состояний;
- энергия физической системы является функцией текущего состояния;
- небольшое возмущение в заданном состоянии приводит к «соседнему» состоянию. Структура связи этих соседних состояний вместе со структурой энергетической функции определяет энергетическую поверхность.

Задача вычислительной минимизации. В типичной задаче такого рода имеется большое (экспоненциальное) множество C возможных решений. Функция стоимости $c(\cdot)$ - оценивает качество каждого решения; для решения $S \in C$ его стоимость записывается в виде $c(S)$. Цель - поиск решения $S^* \in C$ с наименьшим возможным значением (S^*).

Соседские отношения. Одно решение S' может быть получено незначительным изменением другого решения S. Запись $S \sim S'$ означает, что S' является соседним решением S, а множество соседей S, то есть $\{S' : S \sim S'\}$, будет обозначаться $N(S)$.

Алгоритм локального поиска - высокоуровневая схема.

- Он постоянно поддерживает текущее решение $S \in C$.
- На каждом шаге выбирает соседа S' решения S, объявляет S' новым текущим решением и повторяет процесс.
- На протяжении всего выполнения алгоритма запоминается решение с наименьшей стоимостью.

Суть алгоритма локального поиска - выбор соседских отношений и формулировка правила выбора соседнего решения на каждом шаге. *Соседские отношения* - граф множества всех решений, в котором ребра соединяют соседние пары решений. Алгоритм локального поиска - обход графа.

Локальный поиск в задаче о вершинном покрытии

Имеем граф G = (V, E); множество C возможных решений состоит из всех подмножеств S множества V, образующих вершинные покрытия.

Например, всегда выполняется V ∈ C. Стоимость c(S) вершинного покрытия S равна его размеру; таким образом, минимизация стоимости вершинного покрытия эквивалентна нахождению покрытия с минимальным размером.

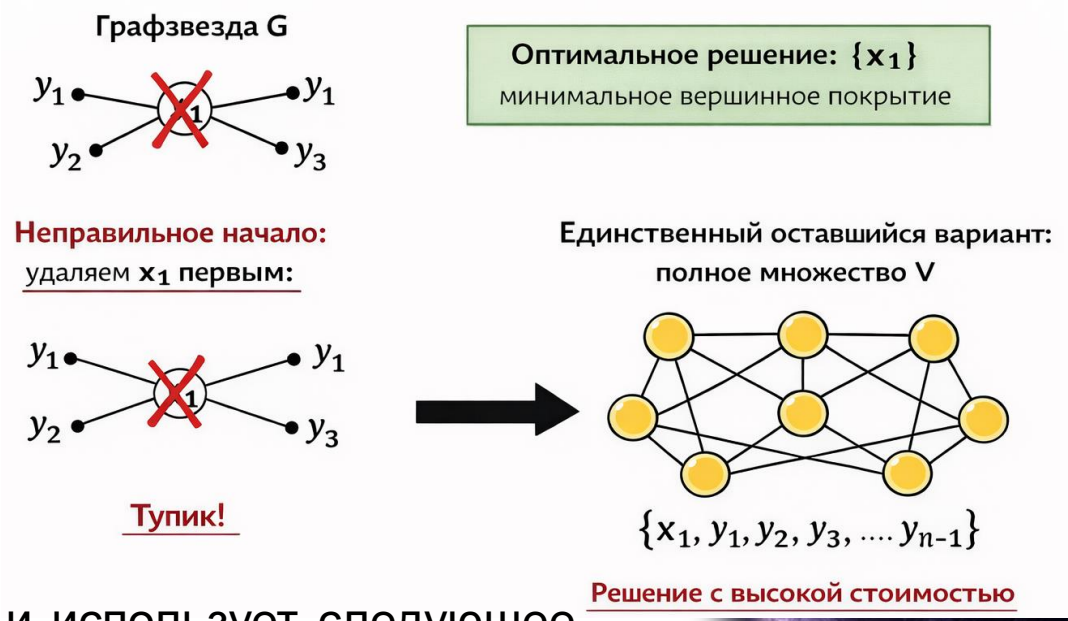
Будем использовать простое соседское отношение: говорим, что S ~ S', если решение S' может быть получено из S добавлением или удалением одного узла.

Таким образом, наши алгоритмы локального поиска будут обходить пространство возможных вершинных покрытий, добавляя или удаляя из текущего решения узел на каждом шаге и пытаясь найти как можно меньшее вершинное покрытие.

Утверждение 28.1. Каждое вершинное покрытие S имеет не более n соседних решений.

Утверждение доказывается попросту тем, что каждое соседнее решение S получается добавлением или удалением узла. Из (28.1) следует, что в процессе выбора можно эффективно исследовать все возможные соседние решения S.

Локальный поиск в задаче о вершинном покрытии



Очень простой алгоритм локального поиска — метод градиентного спуска.

Градиентный спуск начинает с полного вершинного покрытия V и использует следующее правило выбора соседнего решения.

- Обозначим S текущее решение.
- Если ∃ соседнее решение S' со строго меньшей стоимостью, то выбрать соседа, имеющего как можно меньшую стоимость. Иначе завершить работу алгоритма.

Градиентный спуск завершается точно в тех решениях, которые являются локальными минимумами: в таких решениях S, что ∀ соседних S' c(S) ≤ c(S').

Определение очень точно соответствует понятию локального минимума на энергетических поверхностях: это те точки, для которых одношаговое возмущение не улучшает функции стоимости.

Когда метод градиентного спуска работает неправильно? Алгоритм «застревает» в локальном минимуме $\{y_1, y_2, \dots, y_{n-1}\}$, обладающем очень высокой стоимостью в сравнении с глобальным минимумом. В графическом виде - ситуация соответствует «двойной воронке». Глубокая воронка - оптимальное решению $\{x_1\}$, а мелкая воронка - локальный минимум $\{y_1, y_2, \dots, y_{n-1}\}$. Спуск по неправильно выбранному склону, выбранному в самом начале, может привести к неправильному минимуму.

Ситуация легко обобщается.

Локальный поиск в задаче о вершинном покрытии

Допустим, имеется двудольный граф G с узлами x_1, x_2, ..., x_k и y_1, y_2, ..., y_l, where $k < l$ и существует ребро из каждого узла вида x_i в узел вида y_j. Тогда существуют два локальных минимума, соответствующих вершинным покрытиям $\{x_1,..., x_k\}$ и $\{y_1, ..., y_l\}$. Какой из этих минимумов будет обнаружен при выполнении градиентного спуска, зависит исключительно от того, будет ли первым удален элемент вида x_i или y_j.

Интересно: Поверхность ↔ какой граф мог бы ее породить.

Например, какой граф мог бы породить экземпляр задачи о вершинном покрытии с поверхностью вроде «гребенки»? Одним из таких графов мог бы быть n-узловой путь, в котором n — нечетное число, а узлы помечены в порядке v_1, v_2, ..., v_n. Уникальное минимальное вершинное покрытие S* состоит из всех узлов v_i, для которых i четно. Наряду с ним существует множество локальных оптимумов. Например, рассмотрим вершинное покрытие $\{v_2, v_3, v_5, v_6, v_8, v_9, ...\}$, в котором опущен каждый третий узел. Это вершинное покрытие существенно больше S*; при этом из него невозможно удалить ни один узел, сохранив покрытие всех ребер.

Найти минимальное вершинное покрытие S* методом градиентного спуска, начиная с полного вершинного покрытия V, очень трудно: после удаления одного узла v_i с четным значением i возможность нахождения глобального оптимума S* полностью теряется.

Метод создает множество ошибочных путей для локального поиска.

Итог, даже для графов с очень простой структурой градиентный спуск слишком прямолинеен для алгоритма локального поиска.

Алгоритм Метрополиса и имитация отжига

Первые идеи улучшенного алгоритма локального поиска были представлены в работе ***Метрополиса, Розенблата и Теллера (Metropolis, Rosenbluth, Teller, 1953)***. Они рассматривали задачу моделирования поведения физической системы в соответствии с принципами статистической механики. Базовая модель подразумевает, что вероятность нахождения физической системы в состоянии с энергией E пропорциональна функции Гиббса–Больцмана $e^{-E/(kT)}$, где $T > 0$ — температура, а $k > 0$ — константа.

Присмотримся к этой функции. Для любой температуры T функция монотонно убывает по энергии E, поэтому из формулы следует, что физическая система находится в низкоэнергетическом состоянии с большей вероятностью, чем в высокоэнергетическом. Теперь рассмотрим эффект температуры T; при малых T вероятность низкоэнергетического состояния существенно выше вероятности высокоэнергетического состояния. Однако при высокой температуре разность между двумя вероятностями становится очень малой, и система почти с равной вероятностью может оказаться в любом состоянии.

Алгоритм Метрополиса

Метод пошагового моделирования системы при фиксированной температуре T.

- В любой момент времени модель хранит текущее состояние системы и пытается сгенерировать новое состояние, применяя возмущение к текущему состоянию.
- Нас интересуют только состояния системы, «достижимые» из некоторого фиксированного исходного состояния через последовательность мелких возмущений; предполагается, что множество таких состояний C конечно.
- За один шаг сначала генерируется небольшое случайное возмущение в текущем состоянии S системы, приводящее к новому состоянию S'. Пусть E(S) и E(S') обозначают энергии S и S' соответственно. Если E(S') ≤ E(S), то текущее состояние заменяется на S'. В противном случае пусть ΔE = E(S') − E(S) > 0. Текущее состояние заменяется состоянием S' с вероятностью $e^{-\Delta E/(kT)}$, а в противном случае текущим состоянием остается S.

Утверждение 28.2 (Метрополиса без док-ва). Обозначим

$$Z = \sum_{S \in C} e^{-\frac{E(S)}{kT}}.$$

Пусть для состояния S запись $f_S(t)$ обозначает долю первых t шагов, в которых модель пребывает в состоянии S. Тогда ***предел $f_S(t)$*** при стремлении t к ∞, с вероятностью, стремящейся к 1, равен $\frac{1}{Z} \cdot e^{-\frac{E(S)}{kT}}$.Именно такой факт нам и нужен; фактически он означает, что модель проводит приблизительно нужное время в каждом состоянии в соответствии с уравнением Гиббса–Больцмана.

Алгоритм Метрополиса

Работа алгоритма Метрополиса обладает парой свойств, очень полезных для алгоритма локального поиска: он склонен к «нисходящим» перемещениям, но также допускает небольшие «восходящие» перемещения с малой вероятностью. Такой алгоритм сможет двигаться дальше даже при заходе в локальным минимум. Более того, как выражено в (28.2), он глобально смещен в направлении решений с меньшей стоимостью.

Алгоритма Метрополиса для задачи минимизации.

Начать с исходного решения S_0, констант k и T

За один шаг:

Пусть S — текущее решение

Случайно выбрать S' среди соседей S с равномерным распределением Если c(S') ≤ c(S)

Обновить S ← S'

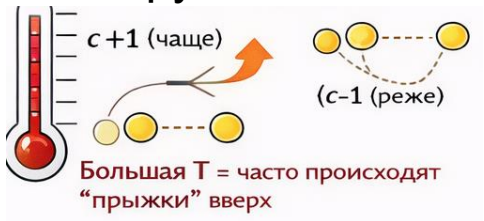
Иначе С вероятностью $e^{-(c(S')-c(S))/(kT)}$

Обновить S ← S'

Иначе Оставить S без изменений

Конец Если

Алгоритм Метрополиса не всегда ведет себя так, как нужно, даже в очень простых ситуациях. По мере работы алгоритма сокращать размер вершинного покрытия становится все труднее и труднее; при приближении к дну воронки начинаются «метания».



Алгоритм Метрополиса

```
void metropolis(void *initial_solution, double k, double T, int max_iterations) {
    srand(time(NULL)); // Инициализация генератора случайных ч
    void *current_solution = initial_solution;
    void *new_solution = NULL;
    for (int i = 0; i < max_iterations; i++) {
        new_solution = get_random_neighbor(current_solution);
        double current_cost = cost(current_solution);
        double new_cost = cost(new_solution);
        if (new_cost <= current_cost)
            copy_solution(current_solution, new_solution);
        else {
            double delta = new_cost - current_cost;
            double probability = exp(-delta / (k * T));
            double random = (double)rand() / RAND_MAX;
            if (random < probability)
                copy_solution(current_solution, new_solution);
        }
        free_solution(new_solution);
    }
}
```

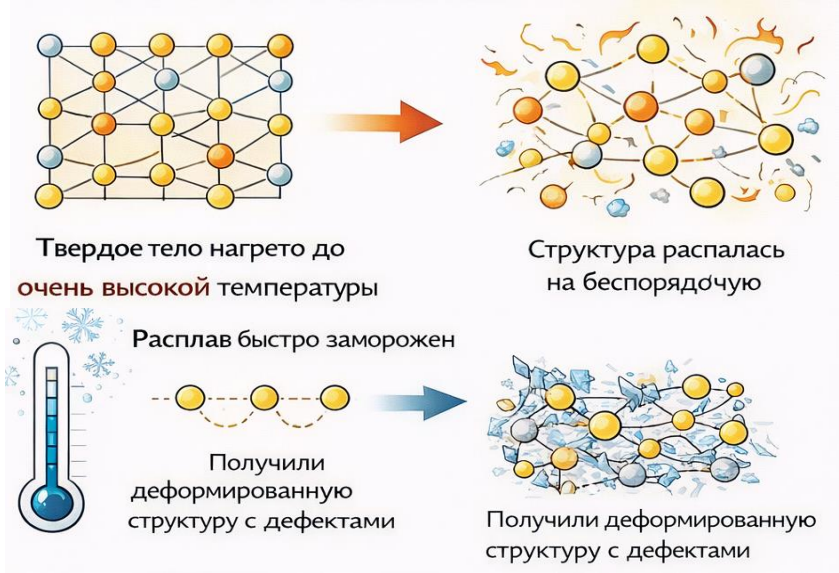
Таким образом, для экземпляра задачи о вершинном покрытии, состоящего из графа-звезды (x_1 соединяется с каждым из узлов y_1, ..., y_{n-1}), мы видим, что алгоритм Метрополиса быстро выходит из локального минимума, возникающего при удалении x_1: соседнее решение, в котором x_1 возвращается обратно, будет сгенерировано и принято с положительной вероятностью. В более сложных графах алгоритм Метрополиса тоже может до некоторой степени исправить неверные решения, принятые в ходе выполнения.

Имитация отжига

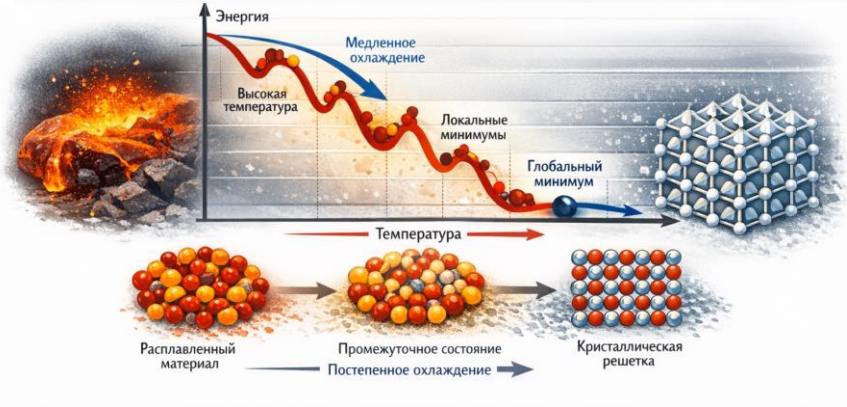
Ни одна из крайних температур — ни очень низкая, ни очень высокая — не является эффективным методом решения задач минимизации в целом.

Аналогия в физических системах:

если взять твердое тело и нагреть его до очень высокой температуры, трудно ожидать сохранения стройной кристаллической структуры, даже если она предпочтительна с энергетической точки зрения; и это можно объяснить большим значением kT в выражении $e^{-E(S)/(kT)}$, с которым огромное количество менее выгодных состояний становится слишком вероятным.



С другой стороны, если взять расплав и очень быстро заморозить его, не стоит также рассчитывать на формирование идеальной кристаллической структуры, скорее вы получите деформированную структуру с множеством дефектов.



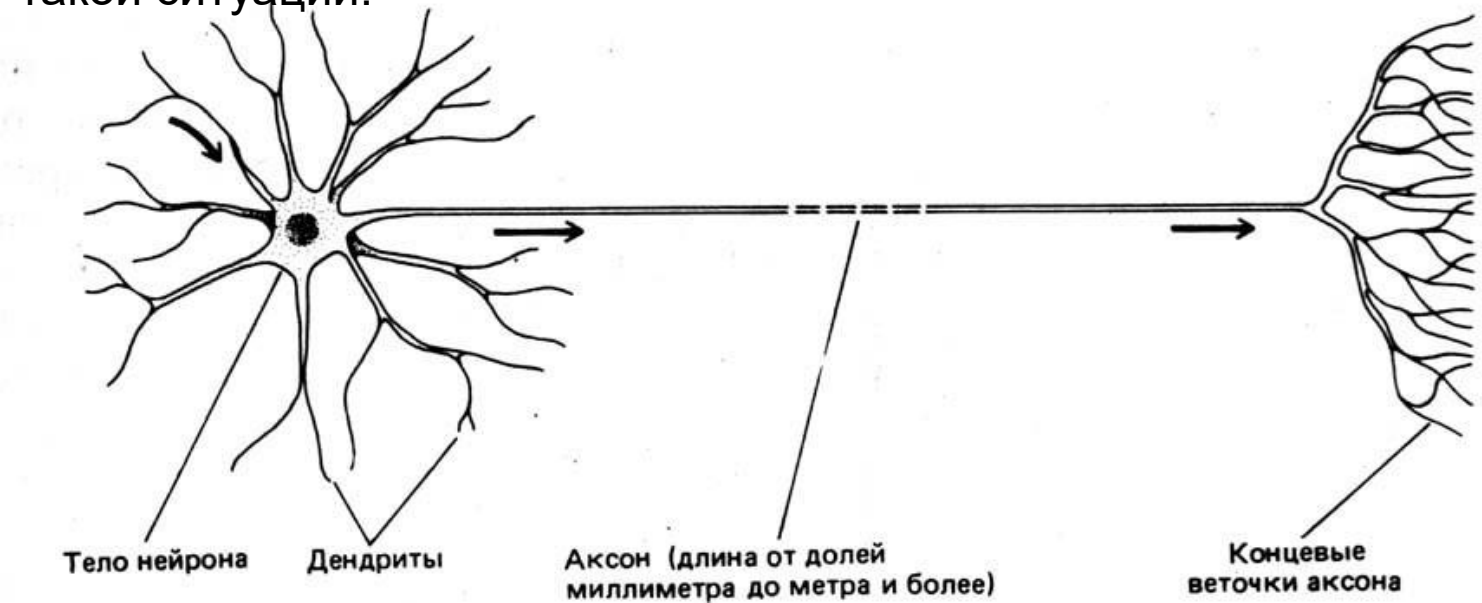
В начале 1980-х годов **Киркпатрик, Гелатт и Веччи (Kirkpatrick, Gelatt, Vecchi, 1983)** размышляли и задали следующий вопрос: как решить эту задачу для физических систем?

В физических системах для перевода материала в кристаллическое состояние используется процесс, называемый отжигом: материал постепенно охлаждается с очень высокой температуры, что дает ему достаточно времени для достижения равновесия на промежуточных убывающих температурах. Таким образом удается избежать энергетических минимумов и достичь глобального оптимума.

Этот процесс на алгоритмическом уровне - известен как имитация отжига.

Применение локального поиска в нейронных сетях Хопфилда

До настоящего момента мы рассматривали локальный поиск как метод поиска глобального оптимума в вычислительной задаче. Однако в отдельных случаях тщательный анализ спецификации задачи показывает, что в действительности требуется найти произвольный локальный оптимум. Следующая задача дает пример такой ситуации.



Задача

Задача ***поиска устойчивых конфигураций в нейронных сетях Хопфилда*** — простая модель ассоциативной памяти, в которой большое количество элементов объединяется в сеть, а соседние элементы пытаются согласовывать свои состояния.

Конкретно **сеть Хопфилда** может рассматриваться как ненаправленный граф G = (V, E) с целочисленным весом для каждого ребра e; каждый вес может быть как положительным, так и отрицательным.

Конфигурация сети S определяется как присваивание значения **−1** или **+1** каждому узлу u; это значение будет называться состоянием s_u узла u.

Смысл конфигурации - каждый узел, представляющий элемент нейронной сети, пытается выбрать одно из двух возможных состояний («да» или «нет»; «истина» или «ложь»), и этот выбор зависит от состояния соседей.

Каждое ребро сети устанавливает требование к своим конечным точкам:

- если u соединяется с v ребром отрицательного веса, то u и v находятся в одинаковом состоянии
- если u соединяется с v ребром положительного веса, то u и v стремятся находиться в противоположных состояниях.

Определение 28.1. Абсолютное значение $|w_e|$ обозначает силу требования; мы будем называть $|w_e|$ абсолютным весом ребра e.

К сожалению, в системе может не быть единой конфигурации, соблюдающей требования всех ребер. Для примера рассмотрим три узла a, b и c, взаимно соединенных ребрами с весом 1. Какую бы конфигурацию мы ни выбрали, два ребра будут находиться в одинаковом состоянии, нарушая требование о противоположности состояний.

Задача

С учетом этого обстоятельства мы немного снизим уровень требований.

В отношении заданной конфигурации ***ребро e = (u, v) называется хорошим***, если устанавливаемое им требование соблюдается состояниями его конечных точек: либо $w_e < 0$ и $s_u = s_v$, либо $w_e > 0$ и $s_u \neq s_v$. В противном случае ребро e называется плохим.

Заметим, что условие «ребро e является хорошим» очень компактно выражается в следующем виде: $w_e s_u s_v < 0$.

Определение 28.2. Узел u в заданной конфигурации называется реализованным, если общий абсолютный вес всех хороших ребер, инцидентных u, по крайней мере не меньше общего абсолютного веса всех плохих ребер, инцидентных u.

Определение 28.3. Конфигурация называется устойчивой, если все узлы в ней реализованы.

Почему такие конфигурации называются «устойчивыми»?

Термин происходит из рассмотрения сети с точки зрения отдельного узла u. Сам по себе узел u имеет выбор только между состояниями −1 и +1; как и все узлы, он хочет учесть как можно больше требований ребер (в соответствии с метрикой абсолютного веса). Допустим, узел u спрашивает: следует ли мне перейти в противоположное состояние? Мы видим, что если u изменяет состояние (при сохранении состояний всех остальных узлов), то все хорошие ребра, инцидентные u, становятся плохими, а все плохие ребра, инцидентные u, становятся хорошими.

Итак, чтобы максимизировать вес хороших ребер, находящихся под его прямым управлением, узел u должен перейти в обратное состояние в том, и только в том случае, если он не реализован.

Основной вопрос:

Всегда ли в сети Хопфилда существует устойчивая конфигурация, и если да, то как ее найти?

Разработка алгоритма

Утверждение 28.3. У каждой сети Хопфилда имеется устойчивая конфигурация, которая может быть найдена за время, полиномиальное по n и $W = \sum_e |w_e|$.

Чтобы доказать (28.3), мы проанализируем простую итеративную процедуру поиска устойчивой конфигурации, которую назовем алгоритмом переключения состояния.

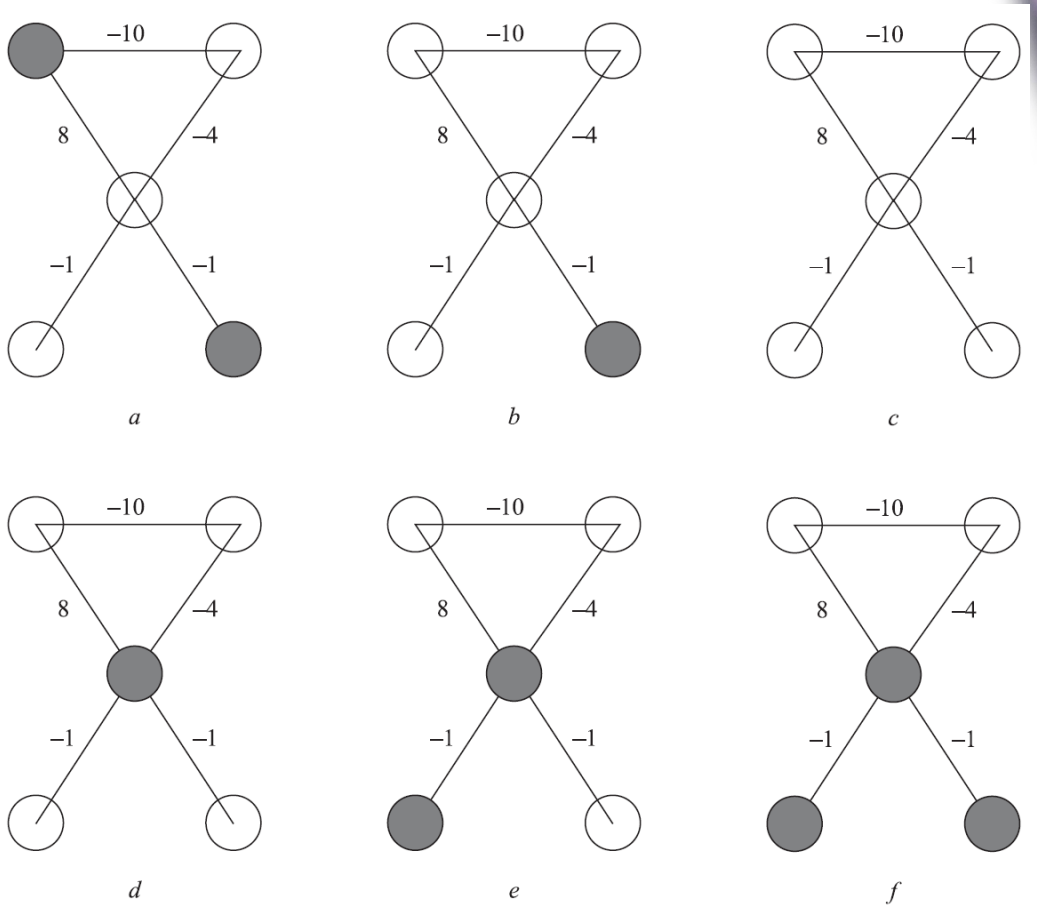
Пока текущая конфигурация не является устойчивой

Должен существовать нереализованный узел

Выбрать нереализованный узел u

Переключить состояние u

Конец Пока



Последовательность выполнения алгоритма переключения состояния для сети Хопфилда из пяти узлов, приводящая к устойчивой конфигурации. (Состояние узлов обозначается черным или белым цветом)

Анализ алгоритма

Докажем, что алгоритм переключения состояния всегда завершается, и приведем границу для количества итераций, необходимых для завершения. Тем самым будет доказано утверждение (28.3).

Ключом к доказательству того, что этот процесс завершается, служит идея:

Нужно найти метрику прогресса, то есть величину, которая строго увеличивается с каждым переключением состояния и имеет абсолютную верхнюю границу. Она может быть использована для ограничения количества итераций.

Существует не очевидная метрика прогресса, которая изменяется с каждым изменением состояния нереализованного узла. А именно: для заданной конфигурации S мы определяем Φ(S) как общий абсолютный вес всех хороших ребер в сети, то есть

$$\Phi(S) = \sum_{good\ e} |w_e|.$$

Очевидно, для любой конфигурации S выполняется Φ(S) ≥ 0 (так как Φ(S) — (так как в крайнем сумма положительных целых чисел) и $\Phi(S) \leq W = \sum_e |w_e|$ случае каждое ребро является хорошим).

Предположим, что в неустойчивой конфигурации S мы выбираем узел u, который является нереализованным, и изменяем его состояние, получая конфигурацию S’.

Что можно сказать об отношениях между Φ(S') и Φ(S)? В

спомним, что при переключении состояния u все хорошие ребра, инцидентные u, становятся плохими; все плохие ребра, инцидентные u, становятся хорошими; а все ребра, для которых u не является конечной точкой, остаются без изменений.

Анализ алгоритма

Если обозначить g_u и b_u общий абсолютный вес соответственно хороших и плохих ребер, инцидентных u, то имеем

$$\Phi(S') = \Phi(S) - g_u + b_u.$$

Но так как узел u был нереализованным в S, мы также знаем, что $b_u > g_u$; а поскольку и b_u, и g_u являются целыми числами, имеем $b_u \geq g_u + 1$. Следовательно,

$$\Phi(S') \geq \Phi(S) + 1.$$

Значение Φ начинается с некоторого неотрицательного целого числа, увеличивается на 1 с каждым изменением состояния и не может превысить W. Следовательно, процесс выполняется не более W итераций, и при его завершении мы должны иметь устойчивую конфигурацию. Кроме того, при каждой итерации нереализованный узел выявляется с использованием полиномиального по n количества арифметических итераций; из этого также следует, что граница времени выполнения полиномиальная по n и W.

Итак, сутью доказательства существования устойчивых конфигураций в конечном итоге оказывается локальный поиск. Сначала мы определили целевую функцию Φ, которую требуется максимизировать. Затем мы изучили поведение простого итеративного алгоритма локального поиска («перевернутую» форму градиентного спуска, так как в данном случае используется задача максимизации); при этом было обнаружено следующее:

Утверждение 28.4. Любой локальный максимум в алгоритме переключения состояния, максимизирующий Φ, является устойчивой конфигурацией.

Вопрос о существовании алгоритма, строящего устойчивые состояния за время, полиномиальное по n и log W, остается открытым.

Аппроксимация задачи о максимальном разрезе с применением локального поиска

Теперь обсудим ситуацию, в которой алгоритм локального поиска применятся для нахождения доказуемой гарантии аппроксимации для задачи оптимизации. Для этого мы проанализируем структуру локальных оптимумов и установим границу качества локально оптимальных решений относительно глобального оптимума. Рассматриваемая задача о максимальном разрезе тесно связана с задачей поиска устойчивых конфигураций для сетей Хопфилда.

Задача

В задаче о максимальном разрезе имеется ненаправленный граф G = (V, E) с положительными целочисленными весами каждого ребра e. Для разбиения (A, B) множества вершин w(A, B) обозначает общий вес ребер, один конец которых принадлежит A, а другой принадлежит B:

$$w(A,B) = \sum_{\substack{e=(u,v) \\ u\in A, v\in B}} w_e .$$

Целью является поиск разбиения (A, B) вершинного покрытия, максимизирующего w(A, B). Задача о максимальном разрезе является NP-сложной в том смысле, что
для взвешенного графа G и границы β принятие решения о том, существует ли разбиение (A, B) вершин G с w(A, B) ≥ β, выполняется с NP-сложностью.

Связь задач нахождения устойчивой конфигурации сети Хопфилда и о максимальном разрезе:

На языке сетей Хопфилда задача о максимальном разрезе представляет собой экземпляр, в котором все веса ребер положительны (а не отрицательны), а конфигурации состояний узлов S естественно соответствуют разбиениям (A, B): узлы имеют состояние –1 в том, и только в том случае, если они принадлежат множеству A, и состояние +1 — в том, и только в том случае, если они принадлежат множеству B.

Целью является такое распределение состояний, при котором как можно большая часть веса приходится на хорошие ребра — те, у которых конечные точки находятся в разных состояниях. В такой формулировке задача о максимальном разрезе направлена на максимизацию величины Φ(S).

Анализ алгоритма

Определение 28.5. Разбиения (A, B) и (A', B') называются ближними соседними решениями, если (A', B') можно получить из (A, B) перемещением одного узла с одной стороны разбиения на другую. Естественно задать два основных вопроса:

- Можно ли сказать что-то конкретное о качестве локальных оптимумов в окружении ближнего соседства?
- Так как ближнее соседство определяется предельно просто, какие варианты соседства могут предоставить более сильные алгоритмы локального поиска для максимального разреза?

Следующий результат отвечает на первый вопрос, показывая, что локальные оптимумы в ближнем соседстве предоставляют решения, соответствующие гарантированной границе аппроксимации.

Утверждение 28.5. Пусть (A, B) — разбиение, которое является локальным оптимумом для задачи о максимальном разделе в ближайшем соседстве, а (A*, B*) — глобально-оптимальное разбиение. Тогда w(A, B) ≥ 0.5 w(A*, B*).

<u>Доказательство.</u> Пусть $W = \sum_e w_e$. Мы также немного расширим систему обозначений: для двух узлов u и v запись w_{uv} будет обозначать w_e, если существует ребро e, соединяющее u и v, и 0 в противном случае. Для любого узла u ∈ A должно выполняться условие

$$\sum_{u \in A} w_{uv} = \sum_{v \in B} w_{uv},$$

так как в противном случае узел u должен быть перемещен на другую сторону разбиения, и разбиение (A, B) не будет локально оптимальным.

Анализ алгоритма

Просуммируем эти неравенства для всех u ∈ A; любое ребро, оба конца которого принадлежат A, будет находиться в левой части ровно двух таких неравенств, тогда как любое ребро, один конец которого принадлежит A, а другой принадлежит B, будет находиться в правой части ровно одного из таких неравенств. Следовательно, мы получаем

$$2\sum_{\{u,v\}\subseteq A} w_{uv} \leq \sum_{u\in A, v\in B} w_{uv} = w(A,B). \quad (28.1)$$

Аналогичные рассуждения можно применить к множеству B, получая

$$2\sum_{\{u,v\}\subseteq B} w_{uv} \leq \sum_{u\in A, v\in B} w_{uv} = w(A,B). \quad (28.2)$$

Сложив неравенства (28.1) и (28.2) и разделив результат на 2, получаем:

$$\sum_{\{u,v\}\subseteq A} w_{uv} + \sum_{\{u,v\}\subseteq B} w_{uv} \leq w(A,B). \quad (28.3)$$

В левой стороне неравенства (28.3) учитываются веса всех ребер, не переходящих из A в B; следовательно, если прибавить w(A, B) к обеим сторонам (28.3), левая сторона станет равна W. Правая сторона превращается в 2w(A, B), поэтому имеем W ≤ 2w(A, B), или w(A, B) ≥ 0.5 W. ■

Анализ алгоритма

Обратите внимание: в доказательстве (28.5) мы особенно не задерживались на оптимальном разбиении (A*, B*); в действительности было доказано более сильное утверждение о том, что в любом локально оптимальном решении в ближнем соседстве по крайней мере половина общего веса ребер в графе пересекает разбиение.

Утверждение (28.5) доказывает, что локальный оптимум представляет собой 2-аппроксимацию максимального разреза.

Возможно ли найти локальный оптимум за полиномиальное время?

Пусть (A, B) — разбиение с весом w(A, B).

Определение 28.6. Для фиксированного ε > 0 переключение одного узла будет называться значительным, если оно улучшает значение разреза минимум на $\frac{2\varepsilon}{n}w(A,B)$, где n = |V|.

Пусть алгоритм переключения состояния принимает только значительные переключения и завершается при отсутствии таких переключений, даже если разбиение не локальный оптимум.

Эта версия алгоритма выполняется за полиномиальное время. Можно расширить предыдущее доказательство и показать, что полученный разрез почти не хуже. Достаточно добавить $\frac{2\varepsilon}{n}w(A,B)$ к каждому неравенству, так как нам известно лишь об отсутствии значительных переключений.

Утверждение 28.6. Пусть (A, B) — разбиение, для которого невозможно значительное переключение, а (A*, B*) — глобально-оптимальное разбиение. В этом случае (2 + ε) w(A, B) ≥ w(A*, B*).

Утверждение 28.7. Версия алгоритма переключения состояния, принимающая только значительные переключения, завершается максимум после $O(\varepsilon^{-1} n \log W)$ переключений (в предположении, что веса целочисленны, а $W = \sum_e w_e$.

Выбор соседского отношения

Алгоритмы локального поиска в действительности базируются на двух основных ингредиентах: выборе соседского отношения и правиле выбора соседнего решения на каждом шаге. **Что следует учитывать при выборе соседского отношения?**

(i) Соседское окружение решения должно быть достаточно широким, чтобы алгоритм не застревал в плохих локальных оптимумах.

(ii) Соседское окружение решения не должно быть слишком большим, так как хотим иметь возможность эффективно искать в множестве соседей возможные локальные перемещения.

Пример: задача о двудольном паросочетании.

Простейшее соседское отношение для паросочетаний выглядит так:

- M' является соседом M, если M' может быть получено вставкой или удалением одного ребра в M. В соответствии с этим мы получаем «поверхности» с множеством зубцов, как и в примерах вершинного покрытия; и по этому определению можно получить локально оптимальные паросочетания, имеющие только половину размера максимального сочетания.

Предположим, мы попытаемся определить более сложное (даже асимметричное) соседское отношение: *M' является соседом M, если при создании соответствующей потоковой сети M' может быть получено из M одним увеличивающим путем.*

Что можно сказать о паросочетании M, если оно является локальным максимумом с этим соседским отношением? В этом случае увеличивающего пути не существует, поэтому M в действительности должно быть (глобальным) максимальным паросочетанием. Единственными локальными максимумами являются глобальные максимумы, так что прямой градиентный подъем приведет к максимальному паросочетанию. Следовательно, мы превратили зазубренную поверхность оптимизации в простую воронку.

Алгоритмы локального поиска при разбиении графов

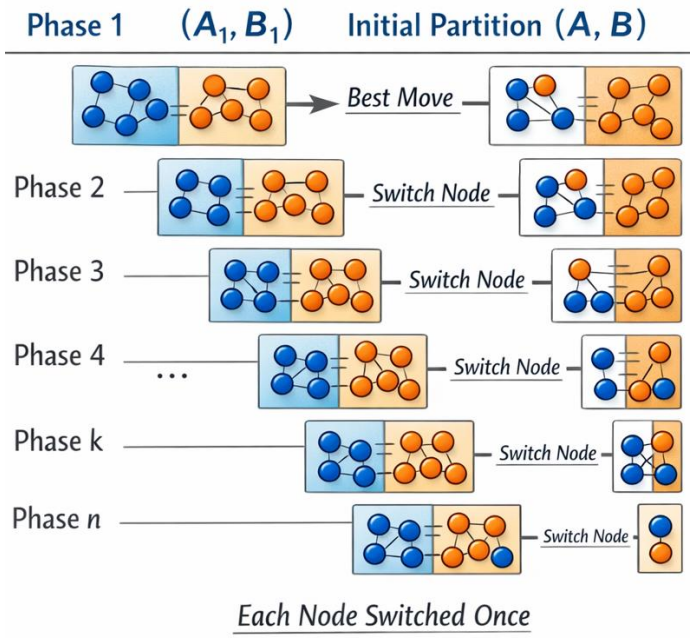
Перейдем к соседским отношениям, порождающим соседские окружения большего размера. Возможно, самым естественным обобщением является соседское окружение с k-переключением для $k \geq 1$: разбиения (A, B) и (A', B') называются соседними по правилу k-переключения, если (A', B') можно получить из (A, B) перемещением не более k узлов с одной стороны разбиения на другую.

Если (A, B) и (A', B') являются соседями по правилу k-переключения, то они также являются соседями по правилу k'-переключения для всех $k' > k$. Следовательно, если разбиение (A, B) является локальным оптимумом по правилу k'-переключения, то оно также является локальным оптимумом по правилу k-переключения для всех $k < k'$.

Но сокращение множества локальных оптимумов посредством повышения величины k обходится дорого: для просмотра множества соседей (A, B) по правилу k-переключения необходимо рассмотреть все $\Theta(nk)$ способов перемещения до k узлов на другую сторону разбиения. Затраты времени становятся неприемлемыми даже при небольших значениях k.

Керниган и Лин (1970) предложили альтернативный способ генерирования соседних решений; он обладает существенно большей вычислительной эффективностью, но все при этом позволяет выполнять крупномасштабные преобразования решений за один шаг.

Эвристикой Кернигана-Лина



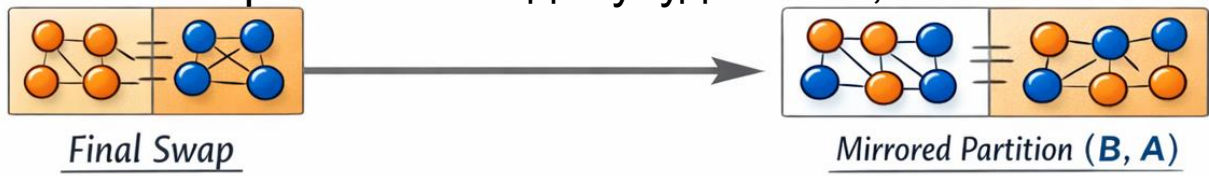
Метод Кернигана и Лина, который мы будем называть ***эвристикой К-Л***, определяет соседей разбиения (A, B) по следующей n-фазной процедуре.

- В фазе 1 выбирается один узел для переключения — так, чтобы значение полученного решения было максимальным. Переключение выполняется даже в случае, если значение решения убывает относительно w(A, B). Изменивший состояние узел помечается, а полученное решение обозначается (A_1, B_1).
- В начале фазы k для k > 1 мы имеем разбиение (A_{k-1}, B_{k-1}), и k – 1 узлов помечены. Один непомеченный узел выбирается для переключения таким образом, что значение полученного решения будет максимальным среди возможных. (даже если в результате значение решения уменьшится.) Переключенный узел помечается, а полученное решение обозначается (A_k, B_k).

После n фаз каждый узел окажется помеченным; это указывает на то, что он переключался ровно один раз. Соответственно последнее разбиение (A_n, B_n) в действительности является зеркальным отображением исходного разбиения (A, B): $A_n = B$ и $B_n = A$.

Эвристика К-Л определяет n – 1 разбиений (A_1, B_1), ..., (A_{n-1}, B_{n-1}) как соседей (A, B). Следовательно, (A, B) является локальным оптимумом по эвристике К-Л в том, и только в том случае, если $w(A, B) \geq w(A_i, B_i)$ для $1 \leq i \leq n - 1$. Видим, что эвристика К-Л определяет очень длинную серию переключений, даже если ситуация на первый взгляд ухудшается, сгенерированное по пути, окажется лучше (A, B).

Выполняется всего лишь n переключений и на каждое тратится время всего O(n).



Классификация на базе локального поиска

Рассмотрим более сложный пример применения локального поиска при разработке аппроксимирующих алгоритмов, связанный с задачей о сегментации изображения, которая рассматривалась среди практических применений сетевых потоков. Более сложная версия сегментации изображения, которая будет рассматриваться здесь, дает пример того, как для получения хорошего быстродействия алгоритма локального поиска приходится использовать довольно сложную структуру соседского окружения для множества решений. Как вы увидите, естественное соседское окружение «переключения состояния» может приводить к очень плохим локальным оптимумам. Для получения хорошего быстродействия мы воспользуемся экспоненциально большим соседским окружением. Одна из проблем больших соседских окружений заключается в том, что последовательный поиск по всем соседям текущего решения для его улучшения становится неприемлемым. Потребуется более сложный алгоритм поиска улучшенного соседа (если он существует).

Задача

Вспомним базовую задачу сегментации изображения. Тогда задача сегментации изображения была сформулирована как задача разметки; наша цель заключалась в пометке (то есть классификации) каждого пиксела как принадлежащего к переднему плану или фону изображения. Это очень простая формулировка задачи и было бы неплохо иметь заняться более сложными задачами разметки, например сегментацией областей изображения в зависимости от их расстояния от камеры.

Рассмотрим задачу разметки с более чем двумя метками. Попутно мы создадим инфраструктуру классификации, возможности применения которой не ограничиваются пикселями изображения. При определении задачи сегментации с двумя метками «передний план/ фон» мы в конечном итоге пришли к следующей формулировке. Имеется граф $G = (V, E)$, в котором V соответствует пикселам изображения, а цель заключается в классификации каждого узла V по двум возможным классам: фону и переднему плану. Ребра представляют пары узлов, которые с большой вероятностью принадлежат одному классу (например, потому что они расположены вблизи друг от друга), и для каждого ребра (i, j) задан штраф $p_{ij} \geq 0$ за размещение i и j в разных классах. Кроме того, имеется информация о вероятности того, принадлежит узел или пиксел переднему плану или фону. Эти вероятности были преобразованы в штрафы за отнесение узла к классу, которому он принадлежит с меньшей вероятностью. Задача заключалась в нахождении разметки узлов, минимизировавшей общие штрафы за разделение и отнесение к классу. Мы показали, что задача минимизации решается посредством вычисления минимального разреза. Далее эта формулировка задачи будет называться бинарной сегментацией изображения.

Задача

Задача классификации/разметки с более чем двумя классами (метками) является NP-сложной, и мы разработаем алгоритм локального поиска, в котором локальные оптимумы являются 2-аппроксимациями для лучшей разметки.

Обобщенная задача разметки:

Имеются граф G = (V, E) и множество L из k меток. *Целью* является пометка каждого узла в V одной из меток из множества L с целью минимизации некоторого штрафа. На выбор оптимальной разметки влияют два конкурирующих фактора. Для каждого ребра (i, j) ∈ E действует штраф за разделение $p_{ij} \geq 0$ за пометку двух узлов i и j разными метками. Кроме того, узлы с большей вероятностью имеют одни метки вместо других. Это условие выражается штрафом за назначение. Для каждого узла i ∈ V и каждой метки a ∈ L действует неотрицательный штраф $c_i(a) \geq 0$ за назначение метки a узлу i. (Эти штрафы являются аналогами вероятностей из задачи бинарной сегментации, не считая того, что здесь они рассматриваются как минимизируемые затраты.) Задача разметки заключается в нахождении разметки f: V → L, минимизирующей общий штраф:

$$\Phi(f) = \sum_i c_i\big(f(i)\big) + \sum_{(i,j)\in E: f(i)\neq f(j)} p_{ij}$$

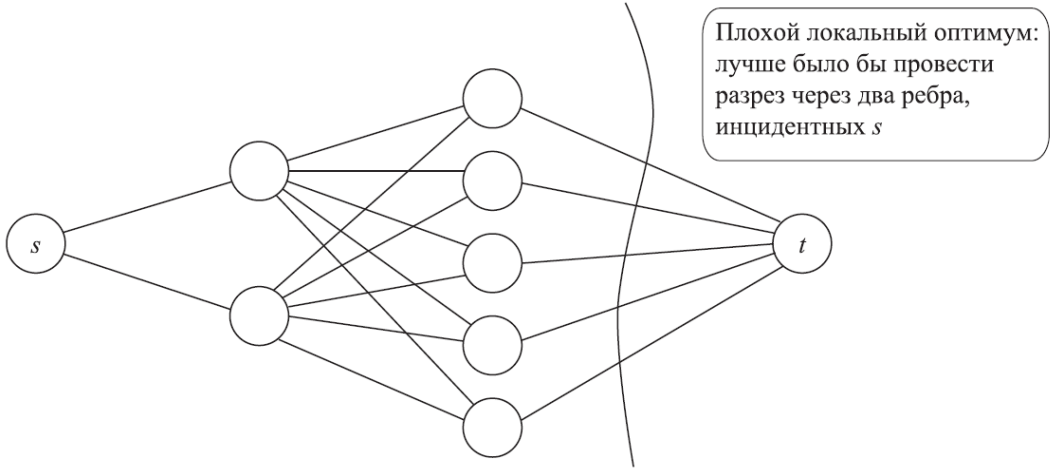
Наша цель — разработать алгоритм локального поиска для этой задачи, в котором локальные оптимумы являются хорошими аппроксимациями оптимального решения. Пример также наглядно покажет, как важно выбирать хорошее соседское окружение для определения алгоритма локального поиска.

Разработка алгоритма

Первая попытка: правило одного переключения

Простейшим и, наверное, самым естественным выбором соседского отношения является правило одного переключения из алгоритма переключения состояния для задачи о максимальном разрезе: две разметки являются соседними, если одну из них можно получить из другой изменением метки одного узла. К сожалению, это соседство может привести к достаточно слабым локальным оптимумам в нашей задаче, даже всего с двумя метками.

Это выглядит несколько странно, потому что правило хорошо работало в задаче о максимальном разрезе. Однако наша задача связана с задачей о минимальном разрезе. Собственно, задача о минимальном разрезе s–t соответствует частному случаю, в котором используются всего две метки, а s и t — единственные узлы со штрафами назначения. Нетрудно увидеть, что этот алгоритм переключения состояния не является хорошим аппроксимирующим алгоритмом для задачи о минимальном разрезе.



Экземпляр задачи о минимальном разрезе s–t, в котором пропускные способности всех ребер равны 1

Разработка алгоритма

Вторая попытка: рассмотрение двух меток за раз

Интересная особенность этого алгоритма - *позволяет каждому решению иметь экспоненциальное количество соседей.*

Сохранение малого размера соседского окружения хорошо работает тогда, когда планируете проводить поиск улучшающего локального шага методом «грубой силы»; на этот раз мы воспользуемся вычислением минимального разреза с полиномиальным временем, чтобы определить, встречается ли улучшение среди exp-многочисленных соседей решения.

Идея локального поиска заключается в применении алгоритма с полиномиальным временем для бинарной сегментации изображения с целью улучшения локальных шагов.

Простейшая реализация этой идеи, которая не всегда дает хорошую гарантию аппроксимации.

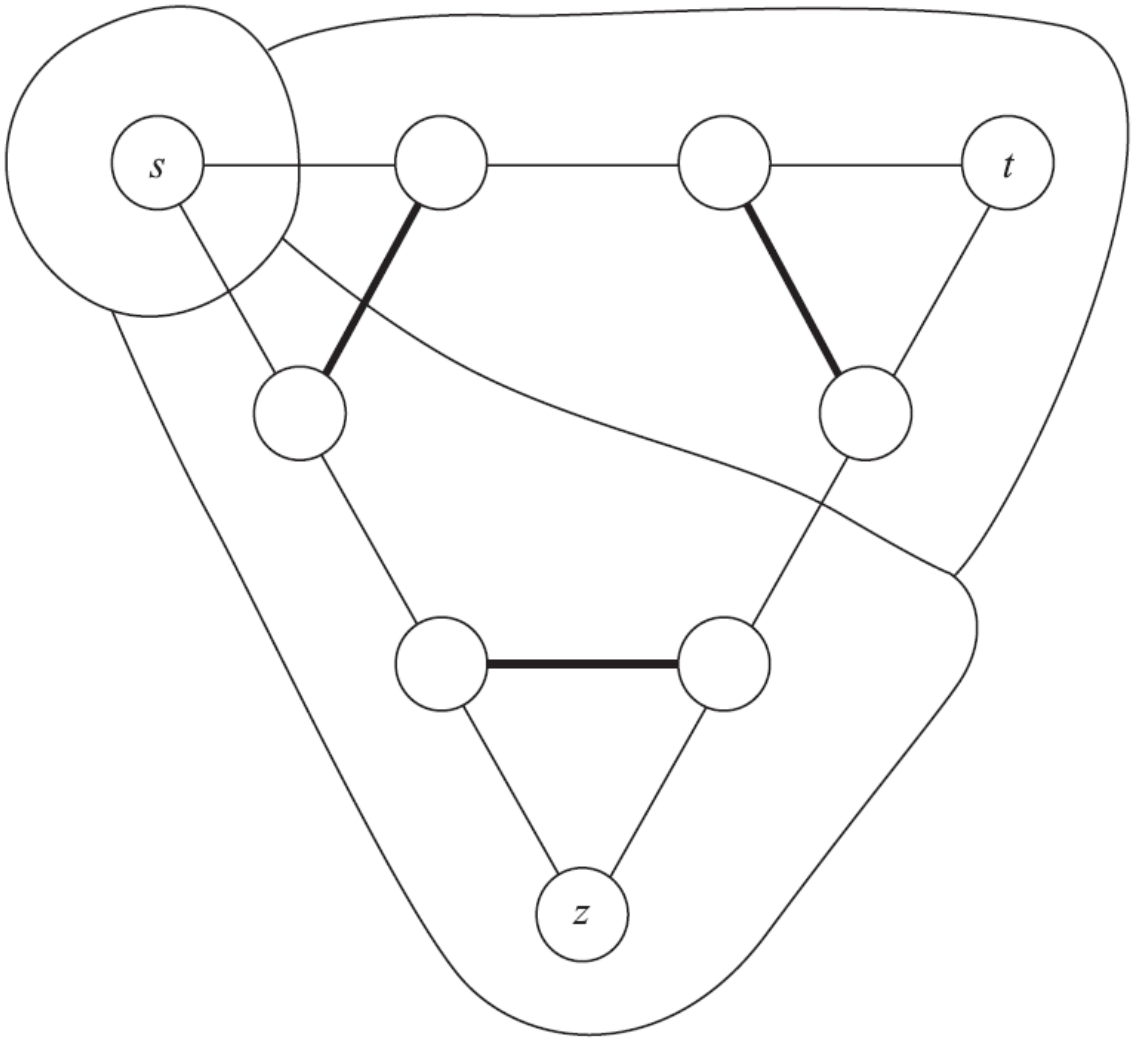
Для разметки f выберем две метки a, b ∈ L и ограничим внимание узлами, имеющими метки a или b в разметке f. За один локальный шаг ∀ подмножество этих узлов может переключить метки из a в b или из b в a. Или в более формальном определении, две разметки f и f ' считаются соседними, если ∃ a, b ∈ L ∀ c ∉ {a, b} ∀ i ∈ V, f (i) = c ⇔ f '(i) = c. Заметим: состояние f может иметь сколь угодно много соседей, так как произвольное подмножество узлов с метками a и b может переключать свою метку.

Утверждение 28.8. Если разметка f не является локально оптимальной для описанного выше соседского окружения, то сосед с меньшим штрафом может быть найден за k^2 вычислений минимального разреза.

Доказательство. Количество пар разных меток меньше k^2, поэтому каждую пару можно проверить по отдельности. Для пары меток a, b ∈ L рассмотрим задачу нахождения улучшенной разметки посредством перестановки меток узлов между a и b. Она в точности совпадает с задачей сегментации для двух меток в подграфе узлов, которым в f назначены метки a или b. Воспользуемся алгоритмом, разработанным для бинарной сегментации изображений, для поиска лучшей из таких измененных разметок. ■

Разработка алгоритма

Например, оно предоставляет оптимальное решение случая с двумя метками. Однако даже с улучшенным окружением локальные оптимумы могут быть плохими, как показано на рисунке. В этом примере есть три узла s, t и z, которые должны сохранить свои исходные метки. Все остальные узлы лежат на одной из сторон треугольника; они должны сохранить одну из двух меток, связанных с узлами на концах этой стороны. Эти требования легко выражаются назначением каждому узлу очень большого штрафа за назначение для неразрешенных меток. Штрафы за разделение определяются следующим образом: тонким ребрам на рисунке соответствует штраф 1, а толстым — большой штраф M. Теперь заметим, что разметка на иллюстрации имеет штраф M + 3, но при этом является локально оптимальной. У (глобально) оптимальной разметки штраф равен всего 3, а для ее получения на рисунке достаточно изменить метки обоих узлов рядом с s.



Плохой локальный оптимум для алгоритма локального поиска, учитывающего только две метки

Поиск хорошего соседа

Чтобы найти лучшего соседа, проверяем каждую метку отдельно. Начнем с метки a. Утверждается, лучшее переназначение (узлы могут перейти на **a** или сохранить текущие метки) находится через **минимальный разрез**. Строится граф G' = (V', E'), аналогично задаче бинарной сегментации. Здесь также мы вводятся **источник** s и **сток** t, но источник s будет представлять метку a, а сток t будет фактически представлять сохранение исходных меток. Идея - найти минимальный разрез в G' и заменить метки всех узлов на стороне s разреза меткой a, тогда как все узлы на стороне t сохранят свои исходные метки.

∀ G в новом множестве V' ∃ соответствующий узел, а в E' добавляются ребра (i, t) и (s, i). Ребро (i, t) имеет пропускную способность $c_i(a)$, так как разрезание ребра (i, t) приводит к размещению узла i на стороне источника, а следовательно, соответствует пометке узла i меткой a. Ребро (i, s) будет иметь пропускную способность $c_i(f(i))$, если f(i) ≠ a, или она будет выражаться очень большим числом M (или +∞), если f(i) = a. Разрезание ребра (i, t) помещает узел i на сторону стока, а следовательно, соответствует сохранению узлом i исходной метки f(i) ≠ a. Большая пропускная способность M предотвращает размещение узлов i с f (i) = a на стороне стока.

В построении для задачи с двумя метками мы использовали штрафы за разделение. Но если i и j находятся на стороне стока, то соединяющее их ребро еще не разрезано, но i и j разделены, если f (i) ≠ f (j).

Узел e всегда может быть размещен так, что разрезается не более одного инцидентного ребра

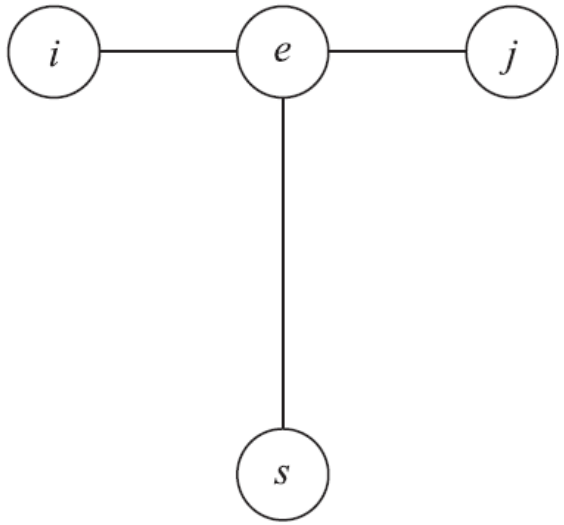


Рисунок. Построение для ребра e = (i, j) с a ≠ f (i) ≠ f (j) ≠ a

Анализ алгоритмов

Чтобы решить эту проблему, мы дополним построение G' следующим образом. Для ребра (i, j), если f (i) = f (j) или один из узлов i или j имеет метку a, в E' добавляется ребро (i, j) с пропускной способностью p_{ij}. Для ребер e = (i, j), у которых f (i) ≠ f (j) и ни один узел не имеет метку a, чтобы правильно закодировать через граф G', что i и j остаются разделенными, даже если находятся на стороне стока, придется действовать иначе. Для каждого такого ребра e в V' добавляется дополнительный узел e, соответствующий ребру e, и ребра (i, e), (e, j) и (e, s) с пропускной способностью p_{ij}.

Утверждение 28.9. Для заданной разметки f и метки a минимальный разрез в графе G' = (V', E') соответствует соседу разметки f с минимальным штрафом, полученному заменой меток подмножества узлов на a. В результате сосед f с минимальным штрафом может быть найден посредством k вычислений минимального разреза, по одному для каждой метки в L.

Утверждение 28.10. Для разметки f и ее соседа f_a

$$\Phi(f_a) - \Phi(f) \le \sum_{i \in V_a^*} \left[c_i\left(f^*(i)\right) - c_i\left(f(i)\right)\right] + \sum_{\substack{(i,j) \\ leaving V_a^*}} p_{ij} - \sum_{\substack{(i,j) in\ or\ leaving V_a^* \\ f(i) \neq f(j)}} p_{ij}.$$

Утверждение 28.11. Для любой локально оптимальной разметки f и любой другой разметки f * — Φ(f) ≥ 2Φ(f *).

Утверждение 28.12. Для любого фиксированного ε > 0 версия алгоритма локального поиска, принимающая только значительные улучшения, завершается за полиномиальное время и приводит к такой разметке f, что Φ(f) ≤ (2 + ε) Φ(f*) для любой другой разметки f*.

Адженда

Локальный поиск

45 минут

Рандомизированные алгоритмы

45 минут

Введение

Идея «случайности» процесса не нова. Она отражена в азартных играх и страховом деле — и то и то происходит из древних времен.

Интуитивно понятные дисциплины, такие как геометрия или логика, изучаются математическими методами уже несколько тысяч лет, область математического изучения вероятности на удивление молода; первые попытки ее серьезной формализации были предприняты в XVII веке. Компьютерная наука существует на много меньшем историческом отрезке, и случайности в ней уделяется внимание с первых дней.

Темы рандомизация и вероятностного анализа проникают во многие области компьютерной науки, включая разработку алгоритмов. Случайные процессы в контексте вычислений:

- Первая рассматривает случайное поведение реального мира: традиционные алгоритмы со случайно сгенерированными входными данными. Такие методы часто называются анализом среднего случая, потому что поведение алгоритма изучается на «средних» входных данных (подвергнутых воздействию некоторого случайного процесса) вместо худшего случая.
- Вторая точка зрения связана со случайным поведением самих алгоритмов: мир всегда поставляет одни и те же входные данные худшего случая, но алгоритму разрешено принимать случайные решения при обработке ввода. Таким образом, рандомизация в этом методе действует исключительно внутри алгоритма и не требует новых допущений относительно природы входных данных.

Первое применение: разрешение конфликтов

Чем может быть полезен алгоритм, принимающий случайные решения?

Рандомизация расширяет возможности используемой модели. Эффективные детерминированные алгоритмы, которые всегда дают правильный ответ, могут рассматриваться как особый случай эффективных рандомизированных алгоритмов, которые дают правильный ответ с очень высокой вероятностью; они также являются особым случаем рандомизированных алгоритмов, которые всегда работают правильно и при этом, как ожидается, выполняются эффективно. Даже при получении худших входных данных алгоритм, выполняющий свою «внутреннюю» рандомизацию, может компенсировать некоторые нежелательные аспекты худшего случая.

Разрешение конфликтов в распределенных системах. Этот пример демонстрирует общий стиль анализа, который будет использоваться во многих дальнейших алгоритмах. В частности, он дает возможность попрактиковаться в основных операциях, относящихся к событиям и их вероятностям, анализу пересечений событий с использованием независимости и объединений событий.

Задача

Допустим, имеется n процессов P_1, P_2, ..., P_n, конкурирующих за доступ к одной базе данных. Будем считать, что время делится на кванты. В одном кванте времени с базой данных может работать не более чем один процесс; если два и более процесса пытаются одновременно обратиться к базе данных, эти процессы «блокируются» до конца кванта. Следовательно, хотя каждый процесс хочет как можно чаще обращаться к базе данных, пытаться обращаться к базе данных всем процессам в каждом кванте бессмысленно; в этом случае все процессы будут постоянно находиться в заблокированном состоянии. Нужен справедливый механизм распределения квантов между процессами, чтобы все процессы имели возможность регулярно работать с базой данных.

Если передача данных между процессами реализуется достаточно просто, можно представить себе всевозможные средства для прямого разрешения конфликта. Но предположим, что процессы вообще не могут передавать информацию друг другу; как определить протокол, по которому они смогут «поочередно» работать с базой данных?

Разработка рандомизированного алгоритма

Рандомизация предоставляет естественный протокол для этой задачи, который определяется очень просто. Для некоторого числа $p > 0$, каждый процесс пытается обратиться к базе данных в каждом кванте с вероятностью **p** независимо от решений других процессов. Итак, если в каком-либо кванте ровно один процесс выдает такое обращение, его попытка завершается успешно; если обращения поступают от двух и более процессов, они блокируются; и если ни одной попытки не было, то квант «пропадает». Такая стратегия, рандомизирующая поведение каждого процесса из множества одинаковых процессов, лежит в основе уже упоминавшейся парадигмы нарушения симметрии: если все процессы действуют синхронно, многократно пытаясь обратиться к базе данных в одно и то же время, никакого прогресса не будет; рандомизация же позволяет «сгладить» конкурентную борьбу.

Анализ алгоритма. Как и во многих других примерах использования рандомизации, алгоритм в этом случае формулируется очень просто; основной интерес представляет прежде всего анализ его эффективности.

Определение основных событий

Анализ вероятностных систем такого рода полезно начать с определения основных событий и их вероятностей.

Первое событие: для заданного процесса P_i и заданного кванта времени t событие A[i, t] обозначает попытку обращения P_i к базе данных в кванте t. Известно, что каждый процесс пытается обратиться к базе данных с вероятностью p, так что вероятность этого события для любых i и t равна Pr[A[I, t]] = p. Для каждого события также существует дополняющее событие, которое означает, что основное событие не произошло; в данном случае дополняющее $\overline{A[i,t]}$ событие означает, что процесс P_i не пытался обратиться к базе данных в кванте t, а его вероятность равна

$$\Pr\left[\overline{A[i,t]}\right] = 1 - \Pr\left[\mathrm{A[I,t]}\right] = 1 - \mathrm{p}.$$

Главный вопрос — удастся ли процессу обратиться к базе данных в заданном кванте. Обозначим это событие S[i, t]. Очевидно, процесс P_i должен хотя бы попытаться обратиться к базе данных в кванте t. Успешное обращение означает, что процесс P_i попытался обратиться к базе данных в кванте t, а все остальные процессы не пытались обращаться к базе данных в кванте t. Следовательно, S[i, t] равно пересечению события A[i,t] с дополняющими событиями $\overline{A[j,t]}$ для j ≠ i:

$$S[i,t] = A[i,t] \cap \left(\bigcap_{j \neq i} \overline{A[j,t]}\right).$$

Все события в этом пересечении независимы по определению протокола разрешения конфликтов.

Определение основных событий

Вероятность S[i, t] вычисляется умножением вероятностей всех событий в пересечении:

$$\Pr[S[i,t]] = \Pr[A[i,t]] \cdot \prod_{j \neq i} \Pr\left[\overline{A[j,t]}\right] = p(1-p)^{n-1}.$$

Мы получили удобное выражение в замкнутой форме для вероятности того, что P_i успешно обратится к базе данных в кванте t.

Вопрос: *как выбрать p для максимизации вероятности успеха?*

Заметим, что вероятность успеха равна 0 для крайних случаев p = 0 и p = 1 (либо процессы вообще не пытаются обратиться к БД, либо каждый процесс пытается обратиться к БД - все процессы блокируются). Функция $f(p) = p(1-p)^{n-1}$ положительна для значений p из диапазона от 0 до 1, а ее производная $f'(p) = (1-p)^{n-1} - (n-1)p(1-p)^{n-2}$ равна нулю только в точке p = 1/n, где и достигается максимум.

Таким образом, вероятность успеха максимизируется при выборе p = 1/n.

Выбирая p = 1/n, получаем $\Pr[S[i,t]] = 1/n\left(1-\frac{1}{n}\right)^{n-1}$.

Утверждение 28.13. (a) Функция $\left(1-\frac{1}{n}\right)^{n}$ монотонно сходится от ¼ к 1/e при увеличении n, начиная с 2.

(b) Функция $\left(1-\frac{1}{n}\right)^{n-1}$ монотонно сходится от 1/2 к 1/e при увеличении n, начиная с 2.

Из (28.13) следует, что $\frac{1}{en} \leq \Pr[S[i,t]] \leq \frac{1}{2n}$, а следовательно, значение $\Pr[S[i,t]]$асимптотически равно $\theta\left(\frac{1}{n}\right)$.

Ожидание успешного обращения со стороны конкретного процесса

Рассмотрим этот протокол с оптимальным значением вероятности обращения p = 1/n. Допустим, нас интересует, сколько времени должно пройти, чтобы процесс P_i успешно обратился к базе данных хотя бы один раз. Из предыдущих вычислений видно, что вероятность успеха в любом отдельном кванте не очень хороша при достаточно больших n. ***Но как насчет серии из нескольких квантов?***

Обозначим $\mathcal{F}$[i, t] «событие неудачи», при котором процесс P_i не добивается успеха во всех квантах с 1 до t. Очевидно, оно представляет собой обычное пересечение дополняющих событий $\overline{S[i,t]}$ для r = 1, 2, …, t. Кроме того, поскольку все эти события независимы, вероятность $\mathcal{F}$[i,t] вычисляется умножением:

$$\Pr\left[\mathcal{F}[i,t]\right] = \Pr[\bigcap_{r=1}^{t}\left[\overline{S[i,t]}\right] = \prod_{r=1}^{t}\Pr\left[\overline{S[i,t]}\right] = \left[1-\frac{1}{n}\left(1-\frac{1}{n}\right)^{n-1}\right]^{t}.$$

Эти вычисления дают значение искомой вероятности, но если и дальше так продолжать, придется иметь дело с чрезвычайно сложными выражениями, поэтому важно переходить на асимптотическое мышление. Вспомните, что вероятность успеха после одного кванта равна $\theta\left(\frac{1}{n}\right)$, а точнее, она лежит в границах от 1/(en) до 1/(2n). Используя приведенное выше выражение, имеем

$$\Pr\left[\mathcal{F}[i,t]\right] = \prod_{r=1}^{t}\Pr\left[\overline{S[i,t]}\right] \le \left[1-\frac{1}{ne}\right]^{t}.$$

Ожидание успешного обращения со стороны конкретного процесса

Теперь заметим, что если задать $t = en$, то мы получим выражение, которое можно напрямую подставить в (28.13). en не будет целым числом, поэтому мы возьмем $t = [en]$ и получим:

$$\Pr\left[\mathcal{F}[i,t]\right] \le \left[1 - \frac{1}{ne}\right]^{|en|} \le \left[1 - \frac{1}{ne}\right]^{en} \le \frac{1}{e}.$$

Это чрезвычайно компактное и полезное асимптотическое утверждение: вероятность того, что процесс P_i не добьется успеха ни в одном из квантов с 1 по |en|,
ограничивается сверху константой e^{-1} независимо от n. Теперь при увеличении t с очень малым коэффициентом вероятность того, что P^i не добьется успеха ни в одном из квантов с 1 по t, резко падает: если задать t = |en| · (c ln n), то получаем

$$\Pr\left[\mathcal{F}[i,t]\right] \le \left[1 - \frac{1}{ne}\right]^{t} \le \left[\left(1 - \frac{1}{ne}\right)^{|en|}\right]^{c\,ln\,n} \le e^{-c\,ln\,n} = n^{-c}.$$

Итак, асимптотически ситуацию можно рассматривать следующим образом:
после $\theta(n)$ квантов вероятность того, что процесс P_i еще не смог обратиться к базе данных, ограничивается константой; а с этого момента до $\theta(n \ln n)$ эта вероятность падает до очень малой величины, ограничиваемой обратной полиномиальной зависимостью от n.

Ожидание успешного обращения всех процессов

Наконец, мы добрались до вопроса, неявно подразумевавшегося в общей постановке задачи: ***сколько квантов должно пройти, чтобы все процессы с достаточно высокой вероятностью обратились к базе данных хотя бы один раз?***

Чтобы ответить на него, будем считать, что выполнение протокола привело к неудаче после t квантов, если какой-то процесс так и не смог обратиться к базе данных. Обозначим $\mathcal{F}_t$ событие неудачи протокола после t квантов; наша цель — найти разумно малое значение t, для которого значение $\Pr[\mathcal{F}_i]$ мало.

Событие $\mathcal{F}_t$ происходит в том, и только в том случае, если происходит одно из событий $\mathcal{F}[i,t]$; мы можем записать

$$F_t = \bigcup_{i=1}^{n} F[i,t]\,.$$

Ранее мы рассматривали пересечения независимых событий, с которыми было очень просто работать; на этот раз речь идет об объединении событий, которые независимыми не являются. Точное вычисление вероятностей подобных объединений может быть очень сложным, и во многих случаях достаточно воспользоваться простой границей объединения, которая гласит, что вероятность объединения событий ограничивается сверху суммой их вероятностей:

Ожидание успешного обращения всех процессов

Утверждение 28.14. (Граница объединения) Для заданных событий $\mathcal{E}_1$, $\mathcal{E}_2$, …, $\mathcal{E}_n$ выполняется

$$\Pr\left[\bigcup_{i=1}^{n} \mathcal{E}_i\right] \leq \sum_{i=1}^{n} \Pr[\mathcal{E}_i] .$$

Обратите внимание: речь не идет о равенстве; однако верхняя граница достаточно хороша в подобных ситуациях, когда объединение в левой части представляет «плохое событие», которого мы пытаемся избежать, и мы хотим ограничить его вероятность в контексте составляющих «плохих событий» в правой части.

Для текущей задачи вспомните, что $\mathcal{F}_i = \cup_{i=1}^{n} \mathcal{F}[i,t]$, а следовательно,

$$\Pr[\mathcal{F}_i] \leq \sum_{i=1}^{n} \Pr[\mathcal{F}[i,t]] \ .$$

Выражение в правой части представляет собой сумму n слагаемых с одинаковыми значениями; чтобы вероятность $\mathcal{F}_t$ была достаточно малой, необходимо убедиться в том, что каждое слагаемое в правой части существенно меньше 1/n. Из предшествующего обсуждения мы видим, что выбор $t = \theta(n)$ недостаточно хорош, поскольку каждое слагаемое в правой части ограничивается только константой. Если выбрать t = |en| · (c ln n), то $\Pr\left[\mathcal{F}[i,t]\right] \leq n^{-c}$ для всех i — именно то, что нужно.

Ожидание успешного обращения всех процессов

А конкретно, выбор t = 2|en| ln n дает нам

$$\Pr[\mathcal{F}_i] \le \sum_{i=1}^{n} \Pr\big[\mathcal{F}[i,t]\big] \le n \cdot n^{-2} = n^{-1}.$$

Таким образом, нам удалось показать следующее:

Утверждение 28.15. С вероятностью не менее $1-n^{-1}$ всем процессам удастся успешно обратиться к базе данных по крайней мере один раз за квантов.

Интересно заметить, что если бы мы выбрали значение t, равное $qn\ln n$ для очень малого значения q (вместо фактически использованного коэффициента 2e), тогда для $\Pr\big[\mathcal{F}[i,t]\big]$ была бы получена верхняя граница, большая n^{-1}, а следовательно, соответствующая верхняя граница для общей вероятности неудачи была бы больше 1, — очевидно, такая верхняя граница полностью бесполезна. Однако, как мы видели, выбирая все большие и большие значения для коэффициента q, можно снизить верхнюю границу $\Pr[\mathcal{F}_i]$ до n^{-c} для любой нужной константы c — это очень малая верхняя граница. Итак, в некотором смысле все «действие» в границе объединения быстро происходит в период, когда $t = \theta(n\ln n)$; с изменением скрытой константы внутри $\theta(\cdot)$ граница объединения переходит от нулевой полезной информации до предоставления в высшей степени сильной границы вероятности.

Для этой задачи (и подобных) мы в действительности находим асимптотически «правильное» значение t, несмотря на использование слабой на первый взгляд границы объединения.

Нахождение глобального минимального разреза

Идея рандомизации естественным образом возникает в приведенном примере — модели с множеством процессов, которые не могут взаимодействовать напрямую. Теперь рассмотрим задачу для графов, в которой тема рандомизации возникает довольно неожиданно, так как для этой задачи существуют вполне разумные детерминированные алгоритмы.

Задача

Для заданного ненаправленного графа G = (V, E) разрез определяется как разбиение V на два непустых подмножества A и B. Ранее, при рассмотрении сетевых потоков, мы работали с похожим определением разреза s–t: тогда для направленного графа G = (V, E) с выделенными узлами источника и стока s и t, разрез s–t определялся как разбиение V на такие множества A и B, что s ∈ A и t ∈ B. Наше новое определение отличается от того: граф стал ненаправленным, и в нем нет ни источника, ни стока.

Для разреза (A, B) в ненаправленном графе G размер (A, B) равен количеству ребер, один конец которых принадлежит A, а другой принадлежит B.

Определение 28.7. Глобальным минимальным разрезом называется разрез минимального размера. Термин «глобальный» указывает на то, что разрешены любые разрезы графа; нет ни источника, ни стока. Таким образом, глобальный минимальный разрез может рассматриваться как естественная характеристика «надежности»; это минимальное количество ребер, удаление которых приводит к потере связности графа. Сначала мы убедимся в том, что методы сетевого потока действительно достаточны для нахождения глобального минимального разреза.

Утверждение 28.16. Существует алгоритм с полиномиальным временем для нахождения глобального минимального разреза в ненаправленном графе G.

Доказательство. Начнем со сходства разрезов в ненаправленных графах и разрезов s–t в направленных графах и с того факта, что нам известен оптимальный способ нахождения последних.

Заданный ненаправленный граф G = (V, E) необходимо преобразовать так, чтобы в нем были направленные ребра, источник и сток. Каждое ненаправленное ребро e = (u, v) ∈ E заменяется двумя противоположно ориентированными направленными ребрами, e' = (u, v) и e'' = (v, u), каждое из которых имеет пропускную способность 1. Обозначим полученный направленный граф G'.

Задача

Теперь допустим, что мы выбрали два произвольных узла s, t ∈ V и нашли минимальный разрез s–t в G'. Легко убедиться в том, что если (A, B) является минимальным разрезом в G', то (A, B) также является разрезом минимального размера в G среди всех разрезов, отделяющих s от t. Но мы знаем, что глобальный минимальный разрез в G должен отделять s от чего-то, так как обе стороны A и B не пусты, и s принадлежит только одной из них. Соответственно мы фиксируем любой узел s ∈ V и вычисляем минимальный разрез s–t в G' для всех остальных узлов t ∈ V − {s}. Это требует n − 1 вычислений направленного минимального разреза, лучшим из которых будет глобальный минимальный разрез графа G. ■

Алгоритм в (28.16) создает сильное впечатление, что нахождение глобального минимального разреза в ненаправленном графе в каком-то смысле является более сложной задачей, чем поиск минимального разреза s–t в потоковой сети, так как нам приходится n – 1 раз вызывать процедуру для решения последней задачи в нашем методе для решения первой. Но оказывается, это всего лишь иллюзия. Серия все более простых алгоритмов, разработанных в конце 1980-х и начале 1990-х годов, показала, что глобальные минимальные разрезы в ненаправленных графах могут вычисляться так же эффективно, как разрезы s–t, и даже еще эффективнее, причем для этого не нужны ни улучшающие пути, ни даже концепция потока.

Кульминационным моментом в этой области стало ***открытие Дэвидом Каргером в 1992 году алгоритма стягивания*** — рандомизированного метода, качественно более простого по сравнению со всеми предшествующими алгоритмами нахождения глобальных минимальных разрезов. В самом деле, алгоритм настолько прост, что на первый взгляд даже трудно поверить, что он действительно работает.

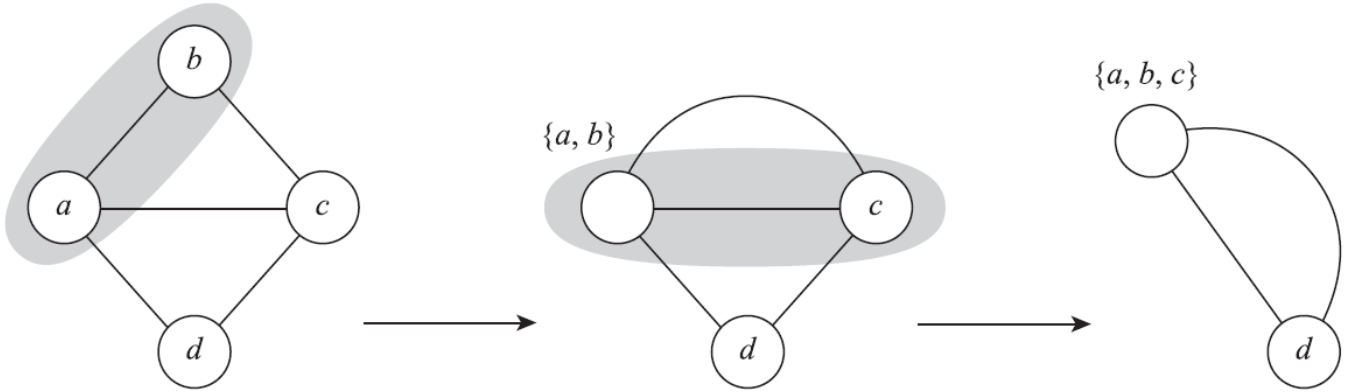
Разработка алгоритма

Описание алгоритма стягивания в простейшей форме.

Эта версия, хотя и работает за полиномиальное время, не принадлежит к числу самых эффективных алгоритмов для глобальных минимальных разрезов. Впрочем, последующие оптимизации алгоритма существенно улучшили его время выполнения.

Алгоритм стягивания работает со связным мультиграфом G = (V, E); это ненаправленный граф, который может иметь несколько «параллельных» ребер между одной парой узлов. Сначала он случайным образом выбирает в G ребро e = (u, v) и «стягивает» его, как показано на рисунке. В результате появляется новый граф G', в котором u и v порождают новый узел w; все остальные узлы сохраняют свою исходную «личность». Все ребра, у которых один конец равен u, а другой равен v, удаляются из G'. Все остальные ребра e остаются в G', но если один конец ребра равен u или v, то этот конец заменяется новым узлом w.

Обратите внимание: даже если в G любые два узла соединены не более чем одним ребром, в G' могут появиться параллельные ребра.



Применение алгоритма стягивания к графу из четырех узлов

Разработка алгоритма

Алгоритм стягивания продолжает рекурсивно выполняться с G', случайно выбирая ребра (с равномерным распределением) и стягивая их. По мере продолжения рекурсии вершины G' должны рассматриваться как суперузлы: каждый суперузел w соответствует подмножеству $S(w) \subseteq V$, «поглощенному» в результате стягиваний, породивших w. Алгоритм завершается при достижении графа G', состоящего из двух суперузлов v_1 и v_2 (предположительно соединенных несколькими параллельными ребрами). Каждый суперузел v_i имеет соответствующее подмножество $S(v_i) \subseteq V$, которое состоит из стянутых в него узлов, и эти два множества $S(v_1)$ и $S(v_2)$ образуют разбиение V. Далее $(S(v_1), S(v_2))$ выводится как разрез, найденный алгоритмом.

Применение алгоритма стягивания к мультиграфу G = (V,E):

- Для каждого узла v хранится множество S(v) узлов, стянутых в v
  - В исходном состоянии S(v) = {v} для каждого v
- Если G содержит два узла v_1 и v_2, вернуть разрез $(S(v_1), S(v_2))$
- Иначе случайно-равномерно выбрать ребро e = (u, v) of G
  - Пусть G' — граф, полученный в результате стягивания e, в котором новый узел z_{uv} заменяет u и v
  - Определить $S(z_{uv}) = S(u) \cup S(v)$
  - Рекурсивно применить алгоритм стягивания к G'
- Конец Если

Анализ алгоритма

Утверждение 28.17. Алгоритм стягивания возвращает глобальный минимальный разрез графа G с вероятностью не менее $1/\binom{n}{2}$.

Общее время выполнения, необходимое для обеспечения высокой вероятности успеха, полиномиально по *n*, так как каждое выполнение алгоритма стягивания занимает полиномиальное время и алгоритм выполняется полиномиальное количество раз.

Утверждение 28.18. Ненаправленный граф G = (V,E) из n узлов содержит не более $\binom{n}{2}$ глобальных минимальных разрезов.

Доказательство - дз

Какие еще можно рассмотреть бы было алгоритмы

- Рандомизированный аппроксимирующий алгоритм для задачи MAX 3-SAT
- Рандомизация принципа «разделяй и властвуй»: нахождение медианы и быстрая сортировка
- Хеширование: рандомизированная реализация словарей
- Нахождение ближайшей пары точек: рандомизированный метод
- Рандомизация кэширования
- Распределение нагрузки
- Маршрутизация пакетов

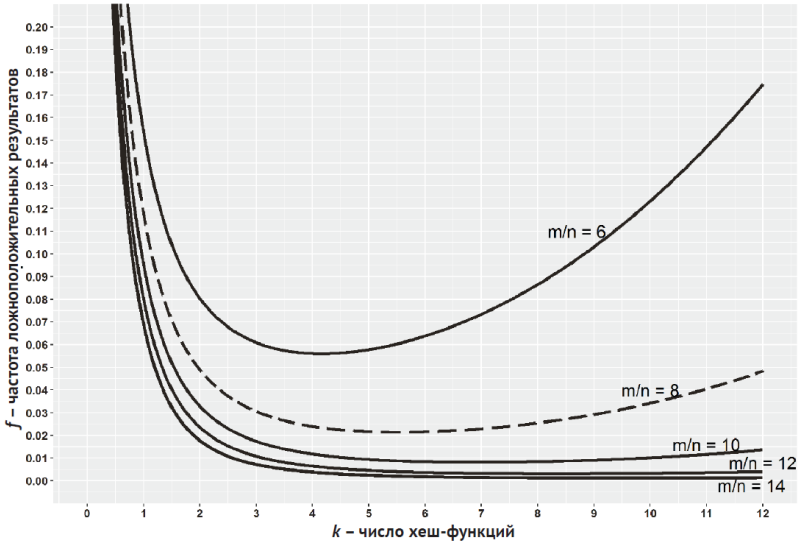
Однако там тоже много тервера и матстата, поэтому просто знайте, что рандомизация в таких алгоритмах дает преимущество в надежности получаемого кода и чаще избегает проблем плохого случая.

В паке будет еще пара задач на рандомизированный алгоритм.

А теперь рандомизация немного с другого боку.

Фильтр «Блюма»

Фи́льтр Блю́ма (англ. Bloom filter) — вероятностная структура данных, сконструированная Бёртоном Блюмом в 1970 году, позволяющая проверять принадлежность элемента к множеству. При этом существует возможность получить ложноположительное срабатывание (элемента в множестве нет, но структура данных сообщает, что он есть), но не ложноотрицательное.

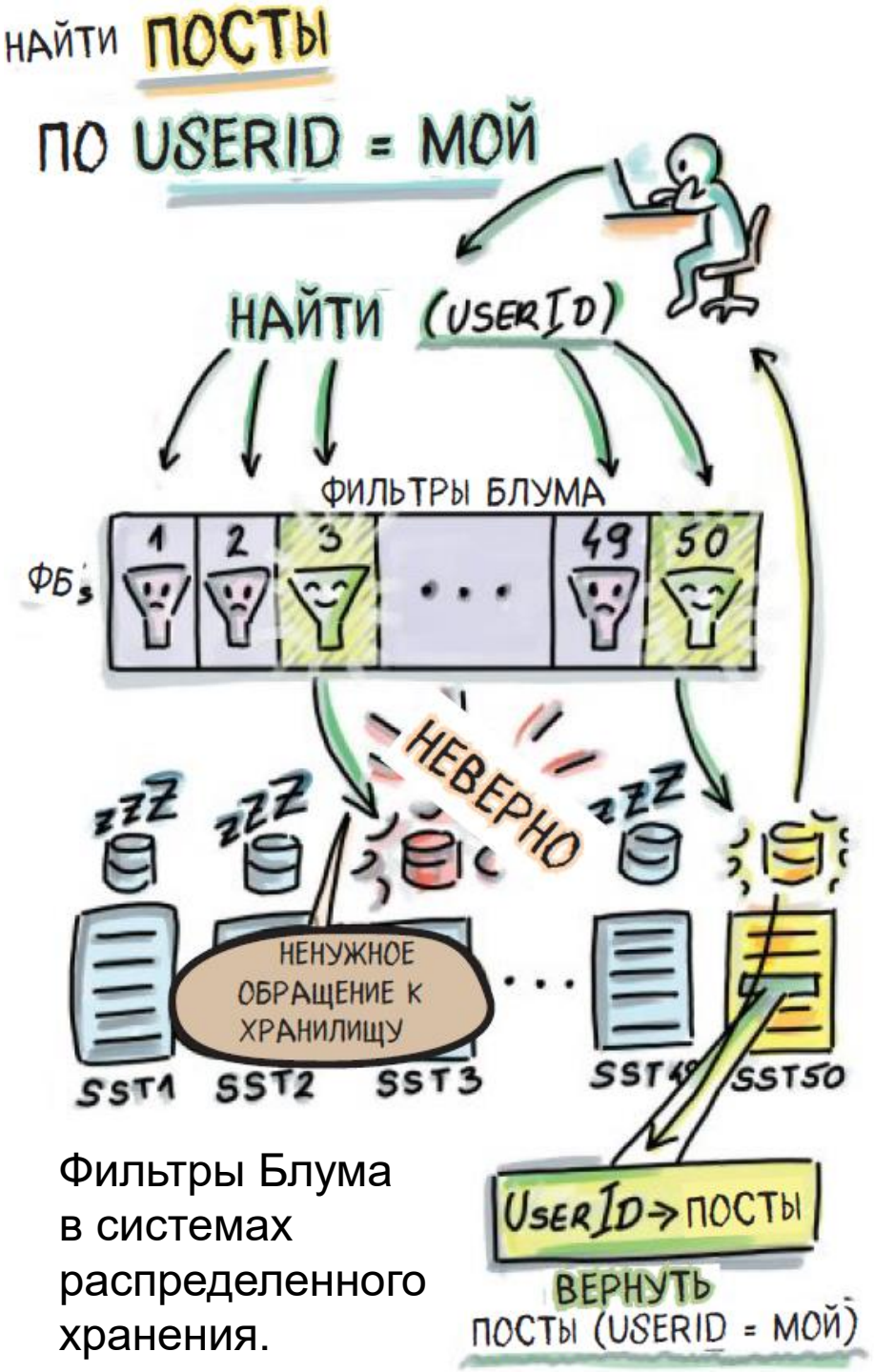


Оценка вероятности ложного срабатывания p как функция от числа вставленных элементов n и размера битового массива m, при оптимальном числе хеш-функций $k = m/n \ln 2$.

Применение Фильтра «Блюма»:

- Google WebTable
- Apache Cassandra

Фильтры Блума можно трактовать как механизм, который поддерживает вставку и поиск таким же образом, как и хеш-таблицы, но использует очень мало пространства.

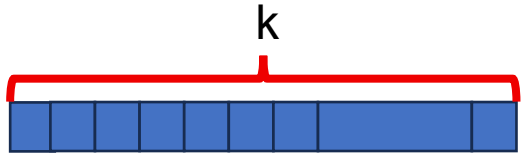


Фильтры Блума в системах распределенного хранения.

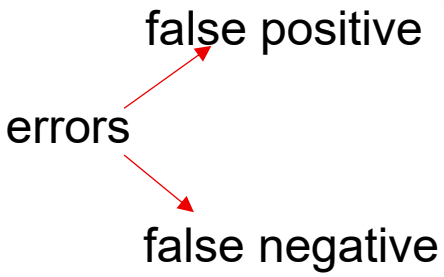
Фильтр «Блюма»

Фильтры Блума состоят из двух главных компонентов:

- битового массива A[0..m-1], все слоты которого изначально установлены равными 0;
- k независимых хеш-функций h_1, h_2, ..., h_k, каждая из которых равномерно случайно отображает ключи в диапазон [0, m – 1].



Имеем хеш-таблицу h: K → {0, …, m-1}
Предположим имеем множество A ⊆ K – все ключи.
Хотим проверять на принадлежность множеству - contains(x).
Хотим вставлять ключи в множество – insert(x)
Если хеш-таблица. Нужно памяти |A| log |K|, меньше чем столько нельзя.

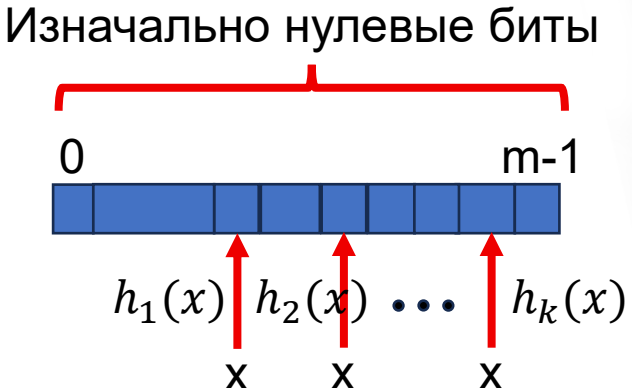


Ослабим требования, некоторые ответы могут быть неправильными.
Ошибки contains(x)

- Могут быть только ложноположительными
- Вероятность ошибки должна быть $P[contains(x)\ \&\ x \notin A] \leq \delta$

Оптимизированная структура для решение такой задачи – Bloom Filter.

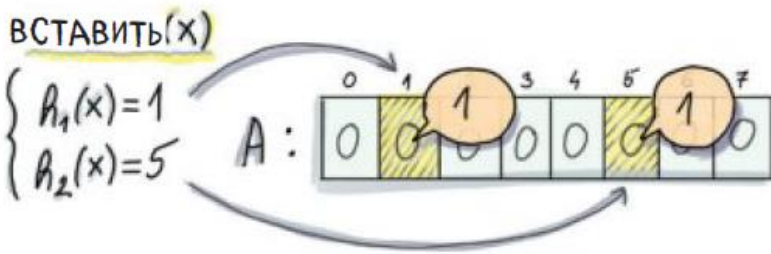
Блюм фильтр не помнит ключей.
Так как довольно необычное добавление – непонятно как удалить, не можем проверить добавление. Функции удаления нет.

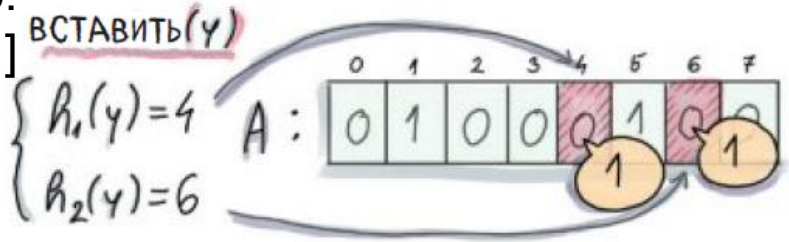


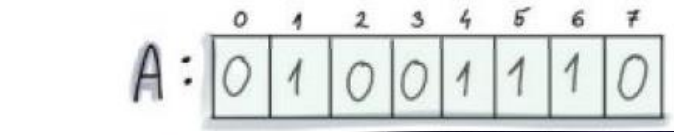
Рандомизированные алгоритмы

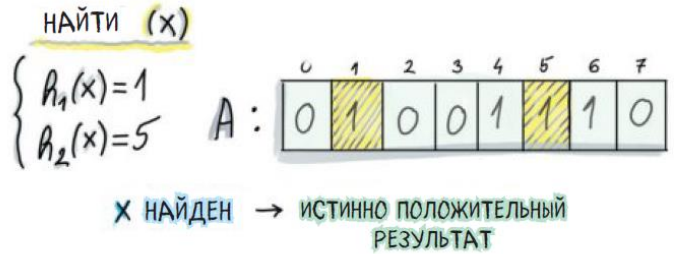
Фильтр «Блюма»

Пустой фильтр Блума имеет m = 8 и k = 2 (две хеш-функции). При вставке элемента x мы сначала вычисляем два хеша на x, первый из которых генерирует 1, а второй 5. Далее мы устанавливаем A[1] и A[5] равными 1.









```
Bloom_insert(x):
    for i ← 1 to k
        A[hi(x)] ← 1
```

Дано: |A| = n, m бит, $h_1, \dots, h_k$

Сколько? $P[contains(x)\ \&\ x \notin A]$

$A = \{a_1, \dots, a_n\}$, $h_i(a_j)$ – количество вариантов (независимых).

Пусть $x \notin A$, тогда $P[contains(x)\,] = P[B[h_1(x)] = 1 \wedge \dots \wedge P[B[h_k(x)] = 1] \approx$

$$\prod_{i=1}^{k} P[B[h_i(x)] = 1] = \left| \begin{array}{c} \text{вероятность коньюнкции} \\ \text{равна произведению} \\ \text{вероятностей} \end{array} \right| = \left(1 - \left(1 - \frac{1}{m}\right)^{nk}\right)^{k} \approx$$

$$\left(1 - e^{\frac{nk}{m}}\right)^{k}$$

Найдем минимум.

$$\ln p = k ln\left(1 - e^{\frac{nk}{m}}\right)$$

$$\frac{d}{dk} \ln p = ln\left(1 - e^{\frac{nk}{m}}\right) + k \frac{-e^{\frac{nk}{m}} \frac{n}{m}}{1 - e^{\frac{nk}{m}}}$$

$$k = \frac{m}{n} ln2$$

$$p_{opt} = 0.5^{\frac{m}{n} \ln 2}$$

```
Bloom_lookup(x):
    for i ← 1 to k
        if(A[hi(x)] = 0)
            return NOT PRESENT
    return PRESENT
```

Фильтр «Блюма» - зачем?

Дано: |A| = n, m бит, $h_1, \dots, h_k$

Сколько? $P[contains(x)\ \&\ x \notin A]$

$A = \{a_1, \dots, a_n\}$, $h_i(a_j)$ – количество вариантов (независимых).

Пусть $x \notin A$, тогда $P[contains(x)] = P[B[h_1(x)] = 1 \wedge \dots \wedge P[B[h_k(x)] = 1] \approx$

$$\prod_{i=1}^{k} P[B[h_i(x)] = 1] = \left| \begin{array}{c} \text{вероятность коньюнкции} \\ \text{равна произведению} \\ \text{вероятностей} \end{array} \right| = \left(1 - \left(1 - \frac{1}{m}\right)^{nk}\right)^k \approx \left(1 - e^{\frac{nk}{m}}\right)^k$$

Найдем минимум.

$$\ln p = k ln\left(1 - e^{\frac{nk}{m}}\right)$$

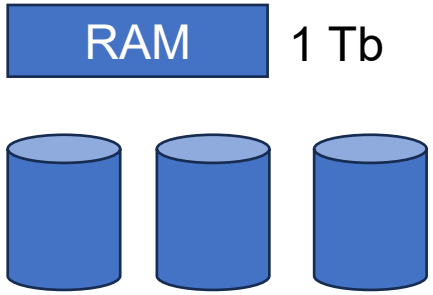
$$\frac{d}{dk} \ln p = ln\left(1 - e^{\frac{nk}{m}}\right) + k \frac{-e^{\frac{nk}{m}} \frac{n}{m}}{1 - e^{\frac{nk}{m}}}$$

$$k = \frac{m}{n} ln2$$

$$p_{opt} = 0.5^{\frac{m}{n} \ln 2}$$

Фильтр «Блюма» - зачем?

Дано: k-value хранилище.



flash/spill
ss-table (file1,2-sort)

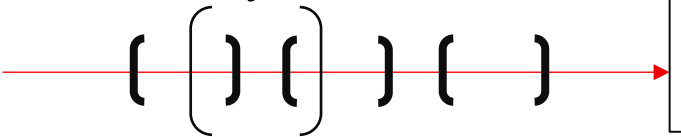
Хотим делать операции contains, insert

2 подхода

1) B-tree
2) LSM

Get – поиск по index
Поверх структуры возникают метаданные, хранятся в оперативной памяти.
Лучший вариант – фильтр Блюма, так как структура ошибается только в 1 сторону

Ищем тут



Чтобы не ходить в 10 файлов, есть функция compaction/

Фильтр «Блюма» - зачем?

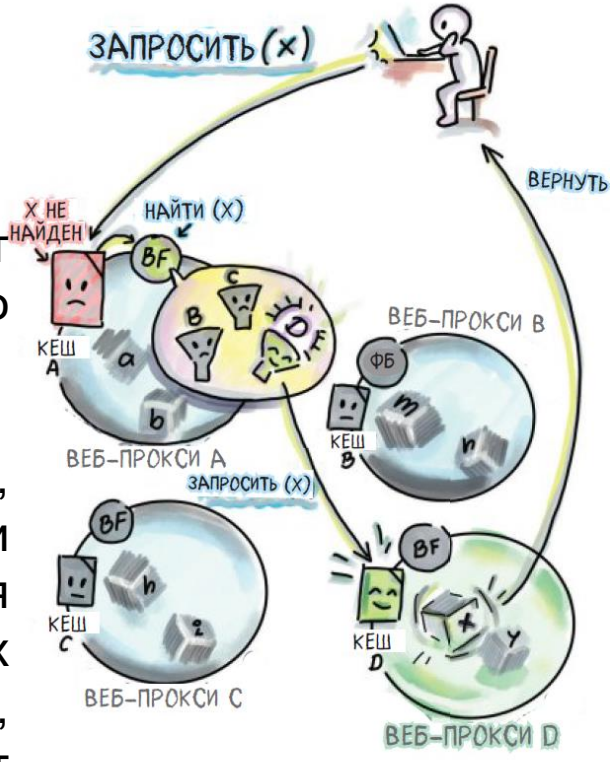
Порционный фильтр – это компактная хеш-таблица, которая обеспечивает пространственно-экономичные выгоды за счет ложноположительных результатов, но также обладает другими преимуществами.

Фильтры Блума в сетях: Squid

Веб-прокси (web proxy) – это процесс, предоставляющий кеш-память для элементов, имеющихся на других серверах, доступ к которым предположительно медленнее или дороже.

Squid – это кеш-память веб-прокси. В веб-прокси кеши используются для уменьшения веб-трафика, поддерживая локальную копию недавно посещенных ссылок, по которым они могут обслуживать запросы клиентов на веб-страницы, файлы и т. д. Один из протоколов предполагает, что каждый прокси также хранит сводную информацию о содержимом кеша соседних прокси и проверяет итоговые данные перед пересылкой любых поисковых запросов соседним прокси. В Squid эта функция реализована с помощью фильтров Блума. Содержимое кеша быстро устаревает, и фильтры Блума эпизодически транслируются между соседями. Поскольку фильтры Блума не всегда актуальны, могут возникать ложноотрицательные результаты, когда фильтр Блума утверждает о присутствии элемента в прокси, но прокси уже не содержит ресурс.

Фильтр «Блюма» - зачем?

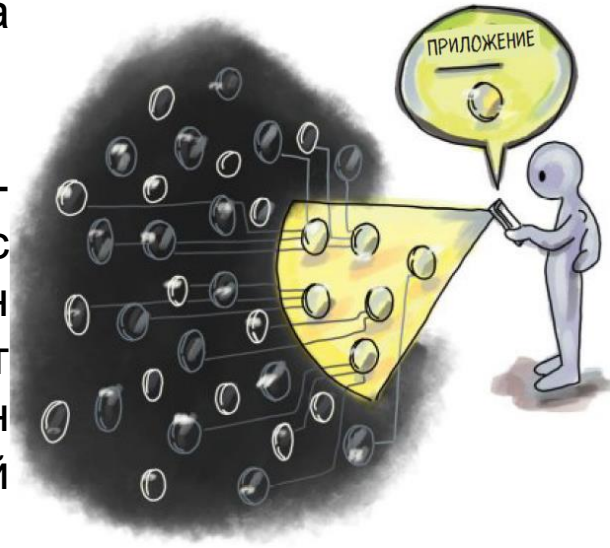


Порционный фильтр – это компактная хеш-таблица, которая обеспечивает пространственно-экономичные выгоды за счет ложноположительных результатов, но также обладает другими преимуществами.

Фильтры Блума в сетях: Squid

Веб-прокси (web proxy) – это процесс, предоставляющий кеш-память для элементов, имеющихся на других серверах, доступ к которым предположительно медленнее или дороже. Squid – это кеш-память веб-прокси. В веб-прокси кеши используются для уменьшения веб-трафика, поддерживая локальную копию недавно посещенных ссылок, по которым они могут обслуживать запросы клиентов на веб-страницы, файлы и т. д. Один из протоколов предполагает, что каждый прокси также хранит сводную информацию о содержимом кеша соседних прокси и проверяет итоговые данные перед пересылкой любых поисковых запросов соседним прокси. В Squid эта функция реализована с помощью фильтров Блума.

Мобильное приложение для биткоинов



Фильтры Блума используются в одноранговых сетях для обмена данными, пример - Биткойн. Непрактично хранить копии всех транзакций в узлах, работающих с устройства с ограниченной памятью, пропускной способностью, поэтому Биткойн предлагает опцию упрощенной верификации платежей, при которой узел может становиться *облегченным узлом*, рекламируя список транзакций, в которых он заинтересован. Облегченные узлы вычисляют и передают полным узлам блумовский фильтр списка транзакций, в которых они заинтересованы.

Оценивание частоты и набросок count-min

Популярны в современных приложениях с интенсивным использованием данных задачи:

- анализа популярности, составление списка бестселлеров, вычисление *k* верхних лидирующих запросов в поисковой системе, составление отчетов о частых парах IP-адресов источник–местоназначение в сети
- Обнаружение аномалий (то есть мониторинг изменений в системах, которые работают в круглосуточном режиме 24/7, таких как сенсорные сети или камеры наблюдения)

Обычно структуры данных ключ-значение, но когда мы сталкиваемся с большим числом несовпадающих элементов?

Допустим, малое число несовпадающих товаров составляет большинство продаж в электронной коммерции. Такого рода распределение (так называемый закон Ципфа) наблюдалось в самых разных областях. Сложность измерения частоты при подобном типе распределения заключается в том, что, с одной стороны, существует законная необходимость решения этой задачи из-за больших вариаций в частоте, а с другой – трудность масштабирования, которая притаилась за углом из-за множества низко- частотных элементов.

Проблемы с масштабируемостью, – это когда ключи представляют собой пары элементов, например пары IP-адресов источник–местоназначение или пары слов о фрагменте текста, где *n* может увеличиваться квадратично по отношению к общему числу несовпадающих IP-адресов (или слов).

Кроме того, эти трудности связаны не только с ограничениями, накладываемыми на пространство.

Majority – задача будет

Задана последовательность ключей (возможны повторения) $A = (a_1, a_2, \dots, a_n)$

$$\exists x \; \mathrm{count}(x) > \frac{n}{2}; \; x = \mathrm{maj}(A)$$

Как найти x за память O(const)

m=1

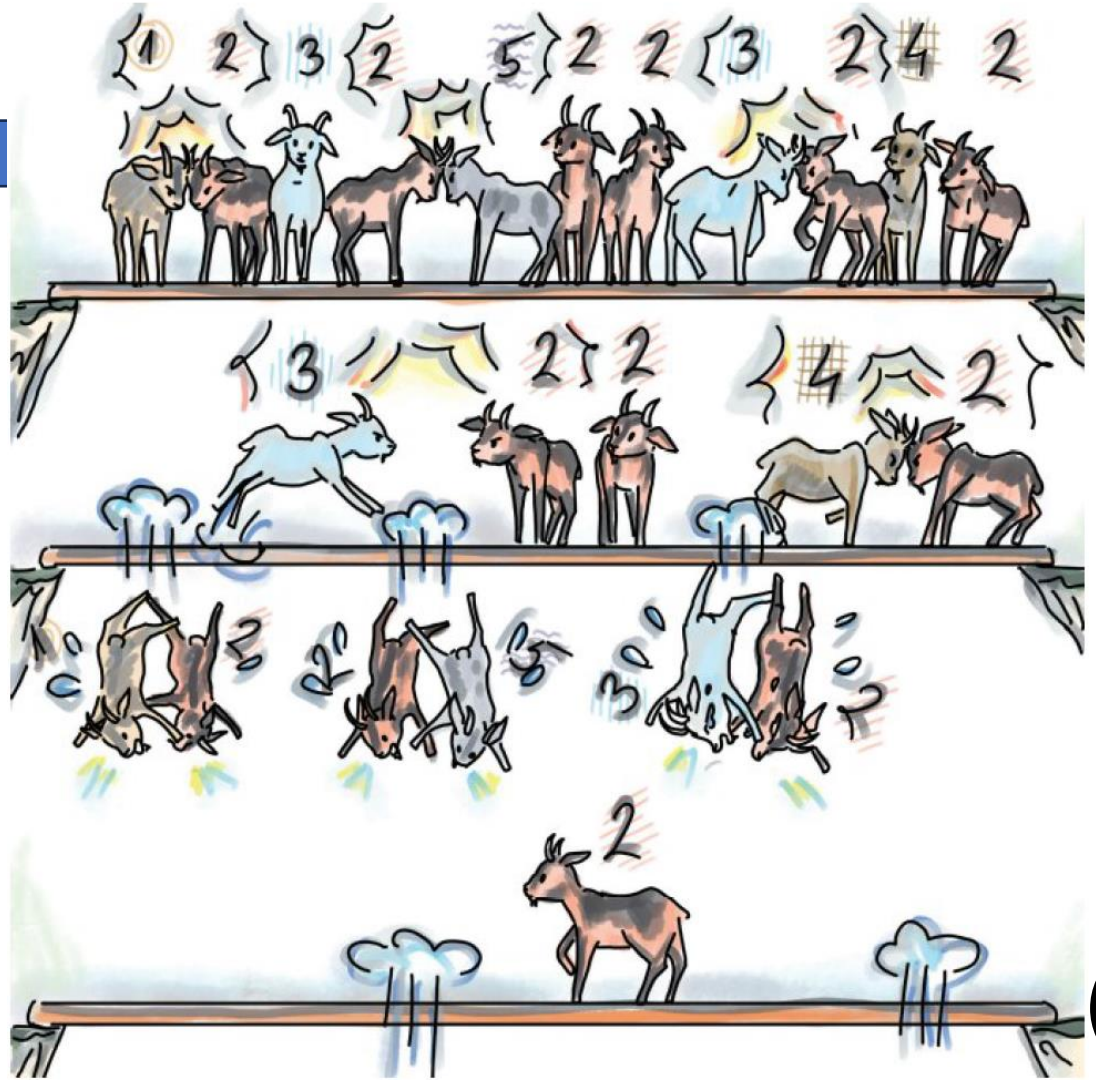
b	c

x=maj(A) →

$$\left(count(x) - \widehat{count}(x)\right) \le \frac{n}{2}$$

$$count(x) > \frac{n}{2}$$

$$\widehat{count}(x) > 0$$



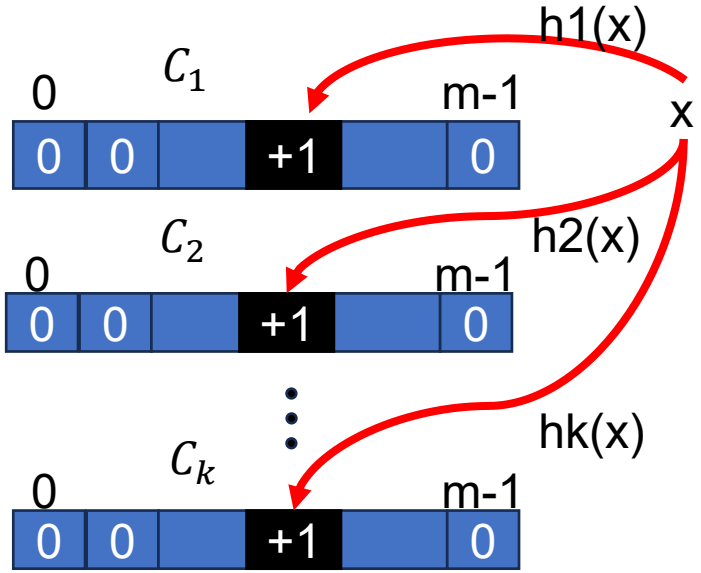
Count-min-sketch

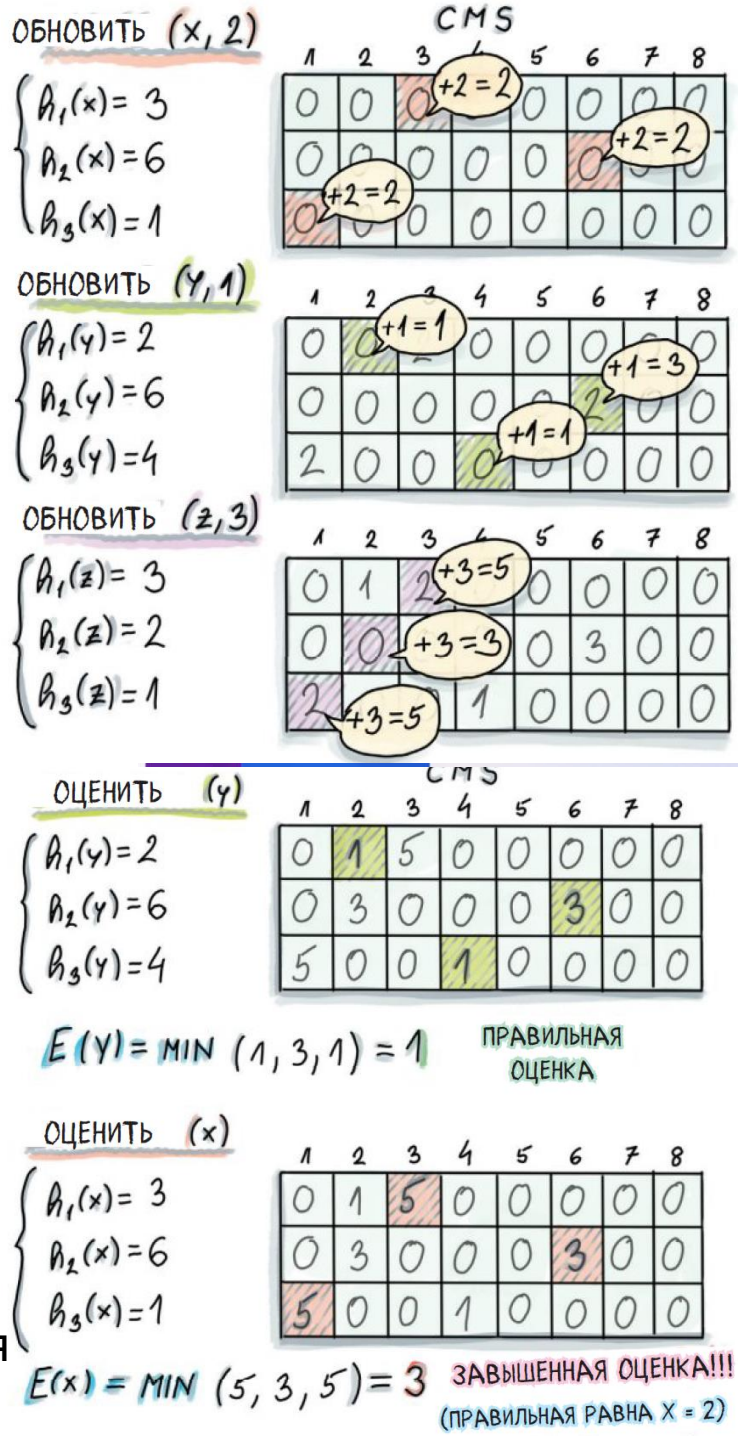
Задана последовательность ключей (возможны повторения) $A = (a_1, a_2, \dots, a_n)$
Хочу знать сколько раз я видел ключ x
$count(x) = \#\{i: a_i = x\}$
Нужна некоторая структура digest (sketch)
Пример – анализ пакетов в маршрутизаторе (памяти есть мало)
Я согласен с тем, что ответ не совсем точен $P\left[\left|\frac{\widehat{count}(x) - count(x)}{n}\right| \geq \varepsilon\right] \leq \delta$
Есть структура данных count-min-sketch которая дает гарантии и имеет space = O(?). Сначала считаем для одной таблицы С потом много Ci.

$$count(x) \leq \widehat{count}(x) = C[h(x)]$$
$$P\left[\widehat{count}(x) - count(x) \geq n\varepsilon\right] = P[\xi \geq n\varepsilon]$$

ξ – случайная величина, поскольку хеш-функция выбирается из универсального семейства

Три операции обновления элементов x, y и z, выполненные на первоначально пустом наброске count-min

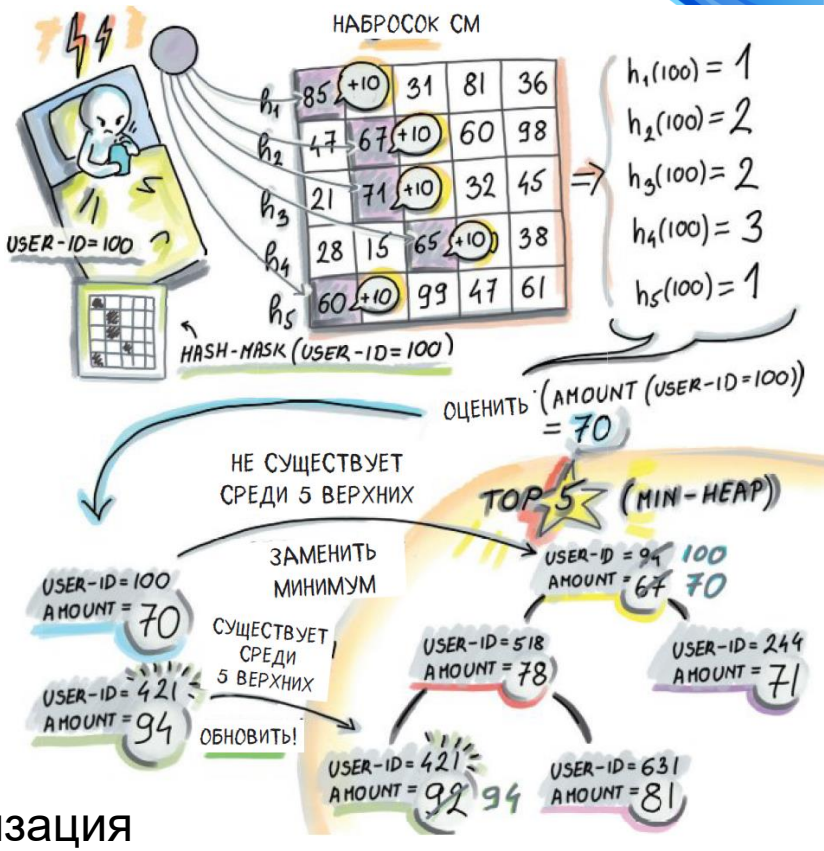




Пример операции оценивания для наброска count-min

Count-min-sketch

Давайте допустим компания выпускает кровати, что одна умная кровать оснащена 100 датчиками и что этот тип умной кровати используется 10^8 потребителями; тогда наша гипотетическая компания ежедневно собирает в общей сложности 10^8 (потребителей) × 3600 (секунд в час) × 24 (часов в сутки) × 100 (датчиков) = 8.6×10^{14} кортежей данных, что приводит к ежедневному объему хранения, исчисляющемуся терабайтами. С каждой покупкой умной кровати поставляется мобильное приложение SleepQuality, которое позволяет пользователям кроватей отслеживать свой сон во временной динамике. Одно из новых приложений отслеживает беспокойство спящих и уведомляет самых беспокойных спящих пользователей о том, что характер их сна отличается от характера сна остальных спящих.



$$\xi = \sum_{\substack{i=1\ldots n \\ a_i \neq x}} 1\{h(a_i) = h(x)\}$$

$$\mathrm{E}\xi = \sum_{\substack{i=1\ldots n \\ a_i \neq x}} P\{h(a_i) = h(x)\} \leq \frac{n}{m}$$

$$\mathrm{P}\left[\widehat{count}(x) - count(x) \geq n\varepsilon\right] \leq \frac{\mathrm{E}\xi}{n\varepsilon} \leq \frac{1}{m\varepsilon} \leq \delta \qquad \boxed{m \sim \frac{1}{\varepsilon\delta}}$$

Минимизация

$$\widehat{count}(x) = \min_i C_i[h_i(x)]$$

$$\mathrm{P}\left[\widehat{count}(x) - count(x) \geq n\varepsilon\right] \leq \left(\frac{1}{m\varepsilon}\right)^k \leq \delta$$

$$\text{Пусть } m = \frac{2}{\varepsilon}, \text{ тогда } k \sim \log_2 \frac{1}{\delta}$$

Посчитаем, что оптимально без доказательства. Память $O(\frac{1}{\varepsilon}\log_2\frac{1}{\delta})$

Misra-Gries

Задана последовательность ключей (возможны повторения) $A = (a_1, a_2, \dots, a_n)$
Хочу знать сколько раз я видел ключ x $count(x) = \#\{i: a_i = x\}$
Я согласен с тем, что ответ не совсем точен $\widehat{count}(x) - count(x) \le \varepsilon n$
C[x] += 1

$$\widehat{count}(x) \le count(x)$$

Утверждаем, что абсолютная погрешность не превышает некоторой величины

$$\forall x \left(count(x) - \widehat{count}(x)\right)(m+1) \le \sum_y \left(count(y) - \widehat{count}(y)\right)$$

1 случай – инкрементирование, очевидно неравенство не меняется 0 = 0
2 случай – декремент и вычеркивание – правая часть растет на m+1, а левая часть растет на m+1 или на 0 если нет ключа в таблице

$$\forall x \left(count(x) - \widehat{count}(x)\right) \le \frac{\sum_y \left(count(y) - \widehat{count}(y)\right)}{m+1} \le \frac{n}{m+1} \le \varepsilon n$$

$$\boxed{m \sim \frac{1}{\varepsilon}}$$

C			
	b_1	C_1	-1
	b_2	C_2	-1
			-1
	⋮		⋮
			-1
	b_m	C_m	-1
a_i	b_{m+1}	1	-1

Нули не хранятся

Зачем нужен Count-min-sketch?

- Count-min-sketch в разных распределениях может оказываться быстрее
- Misra-Gries хранит ключи, что может быть затратно

Оценивание кардинального числа и алгоритм HyperLogLog

Задача определения ***кардинального числа мультимножества*** (множества с дубликатами) получила широкое распространение и возникает во всех областях разработки программного обеспечения, в особенности в приложениях, связанных с базами данных, сетевым трафиком и т. д.

Однако с расширением интернет-служб, в которых миллиарды кликов, поисковых запросов и покупок совершаются ежедневно гораздо меньшим числом несовпадающих пользователей, интерес к этой фундаментальной задаче возрос с новой силой. В частности, существует большой интерес к разработке алгоритмов и структур данных, которые способны оценивать кардинальное число мультимножества за одно сканирование данных и в меньшем объеме пространства.

Использование процедуры оценивания кардинального числа:

•определения числа несовпадающих посетителей, заинтересованных в товаре,

•числа разных пользователей, использующих функции веб-приложения,

•обнаружения внезапных изменений в числе несовпадающих IP-адресов (DoS-атака)

•определение несовпадающих частей контента.

Сегодня растет интерес к разработке алгоритмов, которые могут аппроксимировать точно кардинальное число множества в объеме пространства, существенно меньшем, чем само множество. Один из таких алгоритмов - **HyperLogLog**.

Подсчет числа несовпадающих элементов в базах данных

В SQL используется ключевое слово DISTINCT. Применение SELECT DISTINCT к одному столбцу таблицы - все несовпадающие элементы в этом столбце, SELECT COUNT DISTINCT вернет число несовпадающих элементов в данном столбце. Запросы с операцией COUNT DISTINCT очень распространены, в особенности в электронной коммерции, когда мы хотим получить статистику использования веб-сайта. **Пример:**

```
SELECT COUNT (DISTINCT user_ip_address) WHERE product_id = 9873947
FROM DAILY_VISITS
```

Число несовпадающих IP-адресов (пользователей), обратившихся к товару с id 9873947 в определенный день. На оживленном веб-сайте таблица ежедневных посещений может вырастать до нескольких миллиардов строк. Задержка в связана с операцией сортировки - классическая операция COUNT DISTINCT $O(n \log_2 n)$.

Что еще хуже, даже простые запросы выполняют большое число операций COUNT DISTINCT и GROUP BY на разных столбцах, и сортировка одного столбца не помогает снижать сложность сортировки другого.

Можно было бы задействовать хеш-таблицу - требует линейного пространства.

Для урегулирования трудностей: в SQL Server 2019 есть операция APPROX_COUNT_DISTINCT, Google BigQuery идет еще дальше, и этот приближенный и вероятностный подход используется в операции COUNT_DISTINCT по умолчанию, оставляя операцию EXACT_COUNT_DISTINCT для ситуаций, когда необходим абсолютно точный ответ.

HyperLogLog, первоначально обеспечивает поразительную экономию пространства при обработке наборов данных размером в триллион и низкую частоту ошибки – порядка $O(1/\sqrt{m})$, где m обозначает число ячеек памяти шириной 5 или 6 бит. Одним из распространенных вариантов для m является 2^{14}.

Конструирование HyperLogLog

Суть идеи алгоритма HyperLogLog (HLL) - использование вероятностных и статистических свойств равномерно распределенных случайных битовых строк для угадывания кардинального числа мультимножества.

С этой целью элементы первоначально хешируются в битовые строки: в оригинальной реализации алгоритма HyperLogLog используются 32-битовые хеши, а поздней реинкарнации Google под названием HyperLogLog++ и реализации Redis используются 64-битовые хеши, чтобы вмещать сколь угодно большие кардинальные числа. Хеши достаточно хорошо имитируют случайность.

Имея мультимножество $M = \{a_1, a_2, \dots, a_n\}$ с *n* элементами и *k* несовпадающими элементами (мы не знаем *k*) и используя хеш-функцию $h\colon U \to \{0,1\}^L$, мы генерируем хешированное множество $h(M) = \{h_1, h_2, \dots, h_n\}$, где $h_i = h(a_i)$ с длиной хеша $L = |h_i|$.

Для достаточно большого *L* (например, *L* = 64) каждый несовпадающий элемент будет соотнесен с несовпадающим хешем с высокой вероятностью, так что число несовпадающих хешей тоже будет равно *k* или очень близко.

Конструирование HyperLogLog

Первая примерка: вероятностный подсчет

Самая грубая оценка, именуемая *вероятностным подсчетом*, учитывает битовые регулярности в хеше за счет вычисления ρ_i для каждого хеша h_i таким образом, что

$$\rho_i = (\text{число замыкающих нулей в } h_i) + 1.$$

То есть ρ_i будет обозначать позицию первой единицы, встречающейся справа (если хеш не содержит никаких единиц, то ρ_i = *L* +1). Будем использовать правую сторону. Например, $h_1 = 1100, h_2 = 0111$ и $h_3 = 0000$ соответствующие значения ρ_i равны $\rho_1 = 3, \rho_2 = 1$ и $\rho_3 = 5$. Оценка кардинального числа *E* будет зависеть от $\rho_{\max} = \max(\rho_1, \rho_2, \dots, \rho_n)$, и она равна $E = 2^{\rho_{\max}}$.

```
int p_max = 0;
for (int i = 0; i < n; i++) {
    int h = hash(M[i]);
    int p = __builtin_ctz(h) + 1;
    if (p > p_max) p_max = p;
}
return 1 << p_max;
```

На рис. показан вероятностный подсчет в действии, где *n* = 12, *k* = 7 и окончательная оценка 25 = 32, причем элемент *лимон* существенно влияет на оценку.
Набор данных из 12 элементов хешируется в 16-битовые хеши.
Это не так близко к истине, как нам бы хотелось, но мы только начинаем.

Конструирование HLL

Стохастическое усреднение

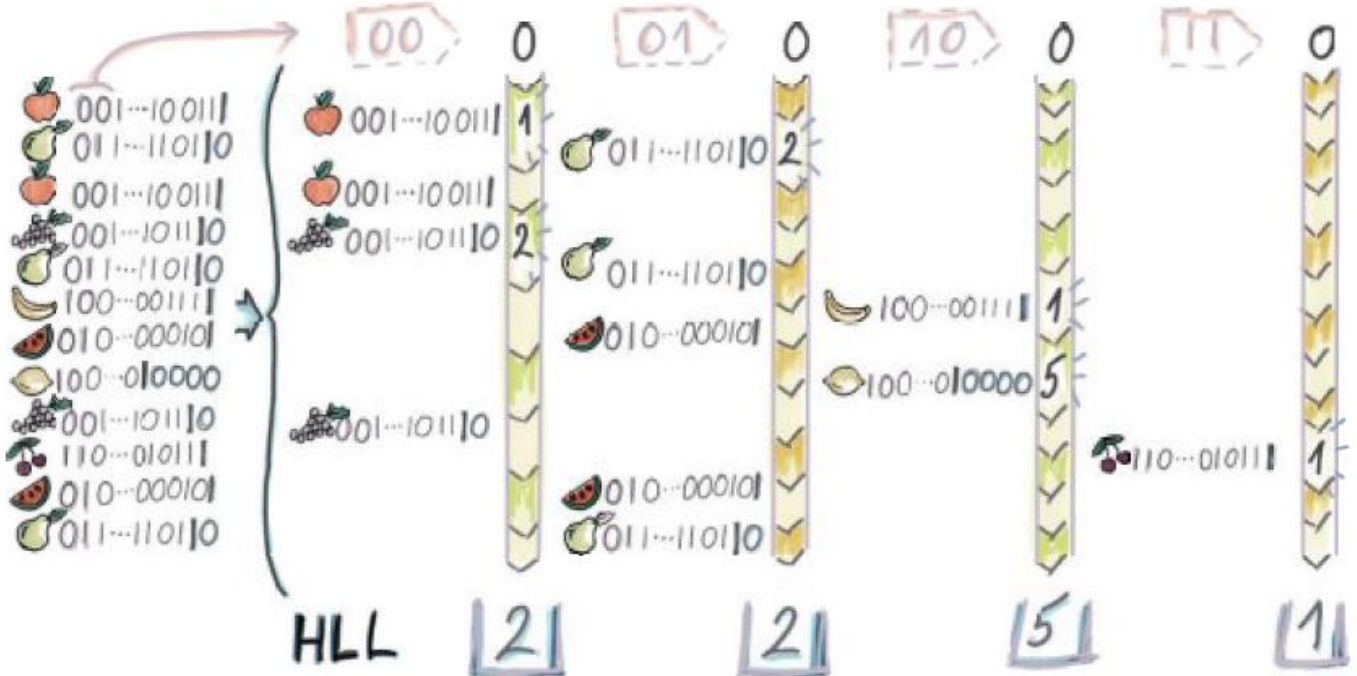
С первым решением есть пара проблем: все оценки являются степенями 2 – невозможно приблизиться к правильному ответу. Метод *стохастического усреднения* разбивает множество хешей равномерно случайно на $m = 2b$ подмножеств примерно одинакового размера, бросая хеши в корзины, задаваемые первыми *b* битами каждого хеша. После того как каждый хеш был назначен корзине, мы выполняем вероятностный подсчет по каждой корзине в отдельности:

```
int m = 1 << b;
int S[m];
memset(S, 0, sizeof(S));
for (int i = 0; i < n; i++) {
    int h = hash(M[i]);
    int p = __builtin_ctz(h) + 1;
    int bucket = h >> (32 - b);  ....
    if (p > S[bucket]) S[bucket] = p;
}
int sum = 0;
for (int i = 0; i < m; i++) sum += S[i];
double avg = (double)sum / m;
return m * (1 << (int)avg);
```

вместо 1-го оценщика $\rho_{\max}$ у нас будет *m* оценщиков $\rho_{i,\max}, 1 \le i \le m$, где $\rho_{i,\max}$ - $\rho_{\max}$ хешей из *i*-й корзины.

На самом деле мы не можем себе позволить *m* хеш-функций и вычислительные затраты на хеширование каждого элемента *m* раз. Далее мы вычислим среднее арифметическое A для *m* оценок и применим его для получения средней оценки по корзинам $E_{\text{корзина}} = 2^A$. Получение совокупной оценки *E* предусматривает учет всех *m* корзин:

$$E = m \times E_{bucket} = m \times 2^{\frac{\sum_{i=1}^{m} \rho_{i,\max}}{m}}$$



Конструирование HLL

Алгоритм LogLog

В алгоритме LogLog стохастическое усреднение используется в сочетании с нормализующей константой $\tilde{a}_m$, которая вводится для устранения смещения из-за систематического завышения оценки, возникающего при оценивании кардинального числа с помощью случайной величины $\rho_{i,\max}$ (максимума геометрических величин параметра 1/2). Следовательно, мы меняем изначальную оценку на формулу:

$$E = m \times E_{bucket} = \tilde{a}_m \times m \times 2^{\frac{\sum_{i=1}^{m} \rho_{i,\max}}{m}},$$

где константа $\tilde{a}m$ параметризуется величиной m и равна $\tilde{a}_m \sim 0.39701 - \frac{2\pi^2+(ln2)^2}{48m}$.

Для большинства практических целей (в частности, при $m \geq 64$) можно использовать $\tilde{a}_m$ = 0.39701. Более подробная информация о формальном

При получении оценки алгоритмом LogLog для текущего примера $\tilde{a}_m = 0.292$, оценка алгоритма LogLog 0.292 × 22.6 ≈ 6.6 - близко к истинному кардинальному числу 7!

Соображения по поводу ошибки и пространства в алгоритме LogLog

относительная ошибка в алгоритме LogLog близко аппроксимируется $1.3/\sqrt{m}$.

Во многих современных реализациях $m \sim 2^{14}$, относительная ошибка ~1.01 %. 2^{14} 8-байтовых целочисленных ячеек занимают всего около 130 Кб, алгоритм LogLog - волшебство! Для корзинных счетчиков нужно всего пять или шесть бит. Если умно посчитать - общие потребности в хранилище в рамках алгоритма LogLog составляют: $O(m \log_2 \log_2(k_{max}/m)) \sim$ 12 Кб.

Конструирование HLL

Алгоритм SuperLogLog

Частоту ошибки в алгоритме LogLog можно попробовать снизить засчет поддержания только процента θ самых низких корзинных значений и основывать оценку на этих $m_\theta = \theta m$ корзинах. Это называется *правилом усечения*.

В аналогичном подходе, именуемом *правилом ограничения*, используются только те корзинные значения, которые не превышают $\lceil \log_2 \left(\frac{k_{max}}{m}\right) + 3 \rceil$, что устраняет выбросы, но и позволяет использовать корзины шириной в $\lceil \log_2 \lceil \log_2 \left(\frac{k_{max}}{m}\right) + 3 \rceil \rceil$ бит. Существуют экспериментальные подтверждения, что при задействовании правил усечения и ограничения частота ошибки падает до $1.05/\sqrt{m}$.

Даже если это лучше по сравнению с базовым подходом на основе вероятностного подсчета, среднее арифметическое в экспоненте все равно может уводить окончательную оценку сколь угодно далеко от среднего, поскольку среднее арифметическое очень чувствительно к выбросам. В нашем последнем усовершенствовании, алгоритме HyperLogLog, для вычисления оценки будет использоваться среднее гармоническое из корзинных значений.

Конструирование HLL

Алгоритм HyperLogLog: стохастическое усреднение вместе с гармоническим средним

Формула гармонического среднего применительно к корзинам, которая представляет наше новое среднее корзинное значение, выглядит следующим образом:

$$E_{\text{корзина}} = \frac{m}{\sum_{i=1}^{m} 2^{-\rho_{i,\max}}}$$

$$E = a_m \times m \times E_{\text{корзина}} = \frac{a_m m^2}{\sum_{i=1}^{m} 2^{-\rho_{i,\max}}}$$

Коэффициент поправки смещения отличается от такого же коэффициента в алгоритме LogLog, и его можно аппроксимировать: $a_m = \frac{1}{2\ln 2(1+\frac{1}{m}(3ln2-1)+O(m^{-2})}$

Для очень больших значений m значение $a_m = \frac{1}{2\ln 2} = 0.72134$.

Примеры α_m:

Статистические доказательства показывают, что относительная погрешность алгоритма HyperLogLog составляет около $1.04/\sqrt{m}$. Потребление пространства как в LogLog: $O(m \log_2 \log_2 k_{max})$. И как в алгоритме LogLog, можно использовать шестибитовые поля.

```
double alpha_m(int m) {
    if (m == 16) return 0.673;
    if (m == 32) return 0.697;
    if (m == 64) return 0.709;
    return 0.7213 / (1 + 1.079 / m);
}
int m = 1 << b, S[1<<b] = {0};
for (int i = 0; i < n; i++) {
    uint32_t h = hash(M[i]);
    int p = __builtin_ctz(h) + 1;
    int bucket = h >> (32 - b);
    if (p > S[bucket]) S[bucket] = p;
}
double sum = 0; int zeros = 0;
for (int i = 0; i < m; i++) {
    sum += 1.0 / (1 << S[i]);
    if (S[i] == 0) zeros++;
}
double E_final, E = alpha_m(m)*m*m/sum;
if (E <= 2.5 * m) {
    if (zeros != 0)
        E_final = m*log((double)m/zeros);
    else
        E_final = E;
} else if (E > (1ULL << 32) / 30.0) {
    E_final = -(1ULL << 32) * log(1 - E /
(1ULL << 32));
} else {
    E_final = E;
}
return E_final;
```

Но как это работает? Мини-эксперимент

Нас интересует величина, на которую нормализующие коэффициенты $\tilde{a}_m$ (в алгоритме LogLog) и a_m (в алгоритме HyperLogLog) повышают точность, а также влияние числа корзин в HyperLogLog на точность и ширину распределения.

Данные всех графиков в этом разделе получены в результате выполнения следующего эксперимента 1000 раз: мы генерируем $N = 2^{16} = 65\,536$ 32-битовых строк, где каждый бит выбирается равномерно случайно. Мы начинаем с равномерно распределенных случайных строк, которые действуют как хеши, потому что мы заинтересованы в получении 1000 наборов хешей с одинаковым (или почти одинаковым) кардинальным числом. Учитывая, что может быть 2^{32} хешей, а размер набора хешей равен 2^{16} , в большинстве экспериментов мы не будем встречать коллизий хешей, и общее число несовпадающих хешей/элементов будет равно размеру набора данных, $N = k = 65\ 536$; эпизодически будет происходить коллизия хешей, но число k несовпадающих элементов никогда не опускается ниже 65 531, что указывает на ничтожную разницу в кардинальных числах между разными экспериментами. Мы раз- работали эксперимент без дубликатов, потому что они не влияют на оцени наших методов, поэтому данный эксперимент также мог бы послужить для демонстрации поведения даже гораздо более крупных наборов хешей, чем 2^{16} , но с 2^{16} несовпадающими элементами.

Мини-эксперимент

На первом графике мы сравниваем следующие ниже методы:

- вероятностный подсчет;
- стохастическое усреднение с ненормализованным средним арифметическим (m = 64);
- стохастическое усреднение с ненормализованным гармоническим средним (m = 64).

Ось x показывает логарифм по основанию 2 кардинального числа; на графике указывается позиция истинного кардинального числа (в 16). Ось y - число проведенных экспериментов.

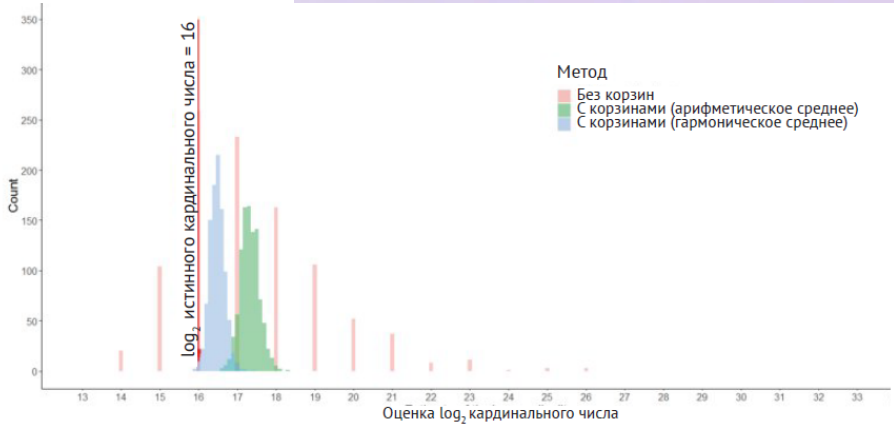
Влияние числа корзин (m)

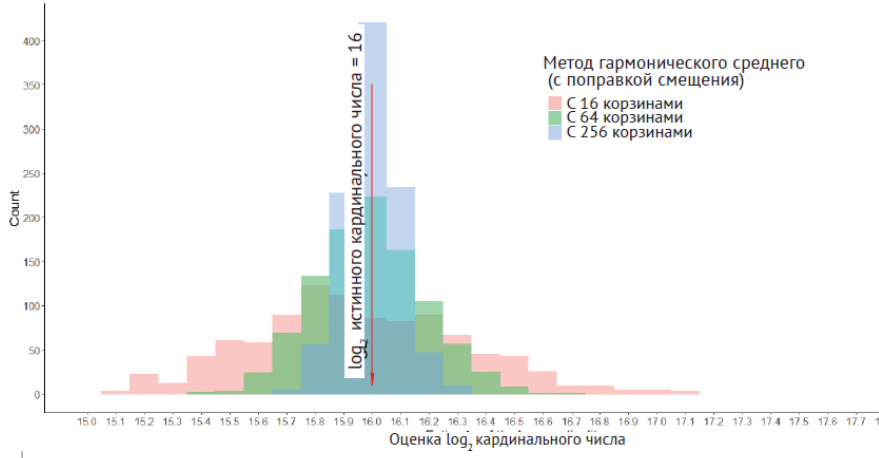
Здесь мы показываем эксперимент с теми же наборами хешей, что и раньше, но на этот раз измеряем эффект от использования трех разных вариантов числа корзин в алгоритме HyperLogLog: m = 16, m = 64 и m = 256. Чем больше корзин, тем с меньшей дисперсией мы сталкиваемся в полученных оценках.

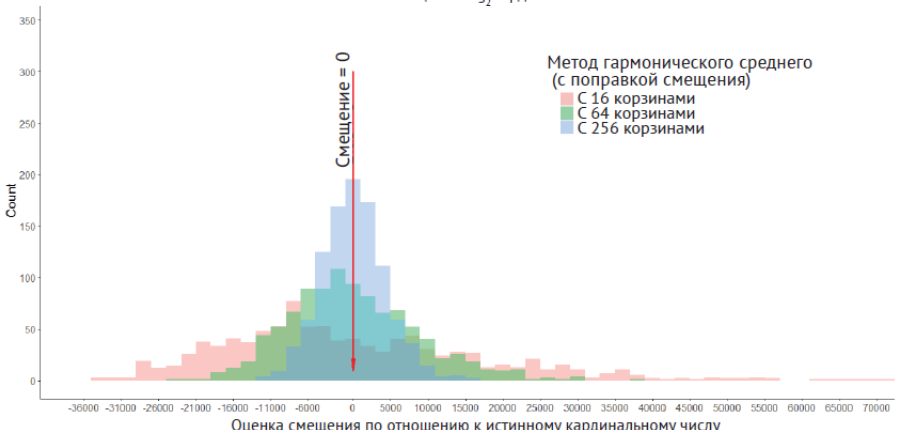
На 3ем рисунке смещение от фактического кардинального числа в каждом эксперименте (ось x – это истинное кардинальное число).

Как отмечалось в оригинальной статье, распределение кардинальных чисел выглядит гауссовым, с более короткими хвостами при большем m.

С учетом стандартной ошибки (или относительной ошибки) алгоритма HyperLogLog в размере 1.04/√m соответственно около 71%, 94.8 % и 99.2 % значений (где значение – это оценка кардинального числа для одного набора данных) будут находиться в пределах σ, 2σ и 3σ долей от истинного кардинального числа.

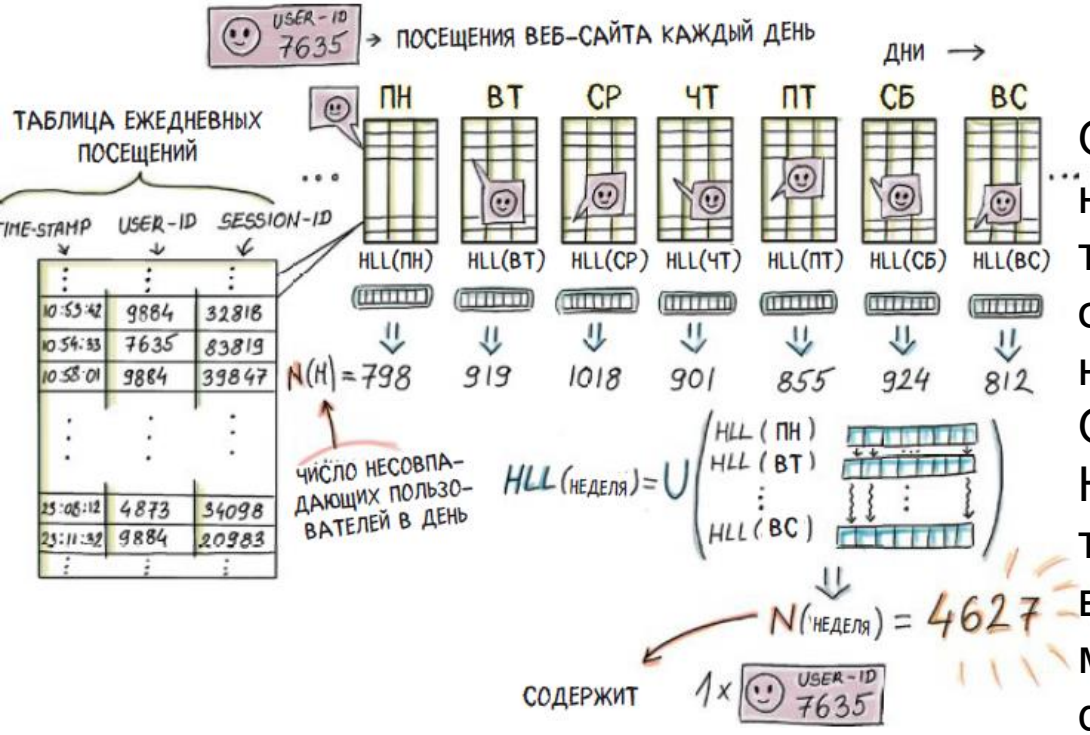


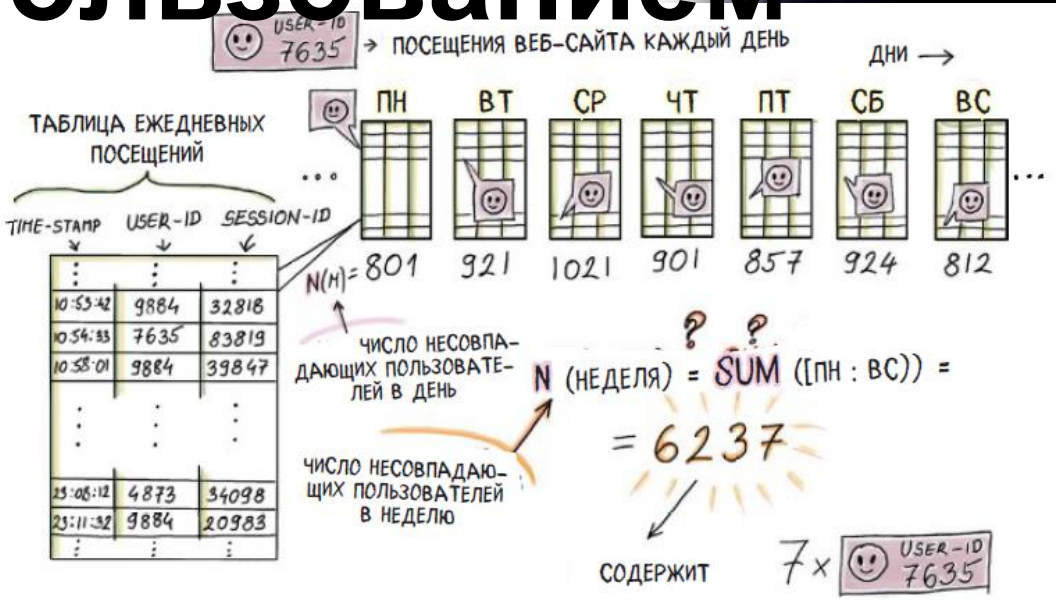




Пример: агрегация с использованием алгоритма HLL

Вернемся к примеру с таблицами ежедневных посещений популярного веб-сайта потребителями. Фотографический веб-сайт Unsplash, на котором размещено большое число изображений и который посещается миллионами пользователей в день, для решения этой проблемы использует алгоритм HyperLogLog.





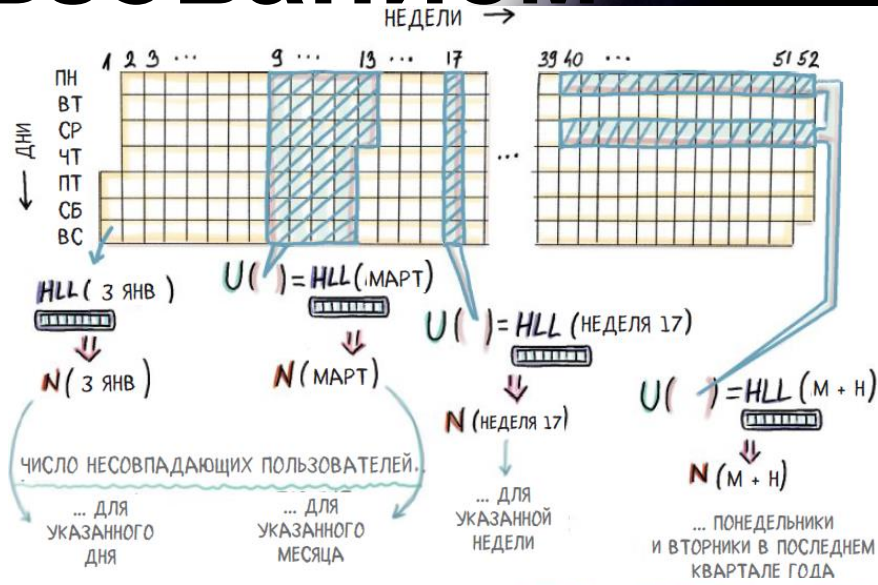
Одна из трудностей, связанных с вычислением числа несовпадающих значений в одном или нескольких столбцах таблицы, заключается в том, что даже если нам волшебным образом будут даны числа несовпадающих значений, это никоим образом не поможет вычислить агрегированное число. Однако если мы сможем поддерживать один массив HyperLogLog в расчете на таблицу ежедневных посещений, тогда мы сможем агрегировать результаты за несколько дней, выполняя операцию объединения между двумя (или более) массивами HyperLogLog того же размера и с той же хеш-функцией.

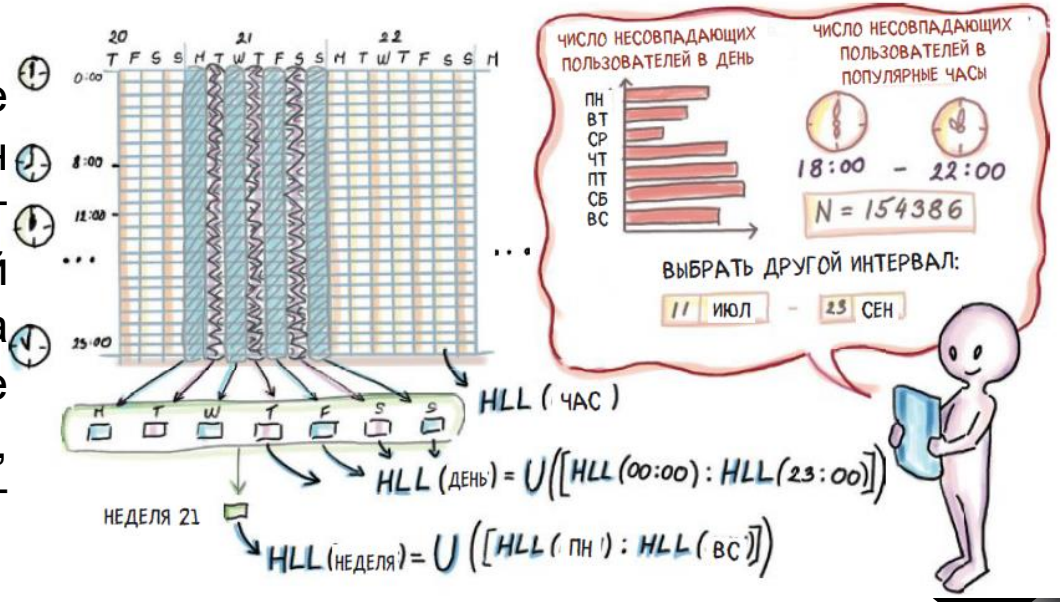
Пример: агрегация с использованием алгоритма HLL

Операция объединения двух массивов HyperLogLog HLL1[1..m] и HLL2[1..m] выполняется за счет создания нового объединенного массива HyperLogLog HLL_UNION[1..m] и назначения max(HLL1[i], HLL2[i]) элементу HLL_UNION[i] для каждого i, $1 \le i \le m$. Например, объединение двух массивов HyperLogLog, значения корзин которых равны (1, 4, 2, 5) и (2, 2, 5, 3), приведет к созданию еще одного массива HyperLogLog со значениями корзин(2, 4, 5, 5).

Что происходит с частотой ошибки при объединении большого числамассивов HyperLogLog?

Относительная ошибка, завися от числа корзин m, после агрегирования остается неизменной, так как число корзин остается неизменным. Относительная ошибка представляет собой *процент* от истинного кардинального числа, который обычно имеет тенденцию увеличиваться с увеличением размера набора данных. Таким образом, даже несмотря на то, что после объединения частота ошибки остается неизменной, константа, на которую ошибка увеличивается, на самом деле растет пропорционально числу несовпадающих элементов.





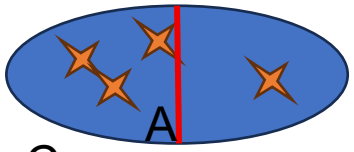
BJKST

Bar-Yossef, **J**ayram, **K**umar, **S**ivakumar и **T**revisan

Задача Distinct Count

$$P\left[\left|d - \tilde{d}\right| \geq \varepsilon\delta\right] \leq \delta, \qquad \varepsilon, \delta > 0$$

HyperLogLog – эмпирически инженерный, BJKST – основан на тех же идеях (теореическая дистилляция HyperLogLog)



$$space = O\left(\frac{1}{\varepsilon^2}\log\frac{1}{\delta}\right)$$

Считаем ключи целые числа $\{0,1\}^w$

A – множество ключей, которые можем помнить наизусть, $|A| \leq \frac{c}{\varepsilon^2}$, если взяли пополам поделили, то $\tilde{d} = 2|A|$.

Хотим хеш-функцию: $\{0,1\}^w \to^h \{0,1\}^w$

h(x) = …000 z(h(x)) = # trailing zeros $A = \{x_i | z(h(x_i)) \geq t\}$ $\tilde{d} = 2^t|A|$

Введем случайную величину $\alpha_i^j = 1[z(h(x_i)) \geq j]$ $E\alpha_i^j = 2^{-j}$ $D\alpha_i^j \leq 2^{-j}$

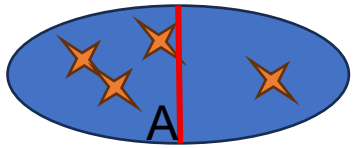
Как понять, сколько уникальных элементов: $|A| = \sum_{i:x_i-distinct} \alpha_i^t =: \beta^t$ $E\beta^t = d2^{-t}$ $D\beta^t \leq d2^{-t}$

$$P\left[\left|\tilde{d} - d\right| \geq \varepsilon\delta\right] = P[|2^t\beta^t - d| \geq \varepsilon\delta] = (\text{из функции вычитается ее матожидание}) = P[|\beta^t - d2^{-t}| \geq \varepsilon\delta 2^{-t}]$$

$$= \sum_{\tau=0}^{w} P[|\beta^t - d2^{-t}| \geq \varepsilon\delta 2^{-t} \wedge t = \tau] \leq (\text{неравенство Чебышева}) \leq \sum_{\tau=0}^{s-1} \frac{D\beta^\tau}{(\varepsilon^2 2^{-\tau} d)^2} + P[t \geq s] = \sum_{\tau=0}^{s-1} \frac{2^\tau}{\varepsilon^2 d} + P[t \geq s]$$

$$< \frac{2^s}{\varepsilon^2 d} + P\left[\beta^{s-1} > \frac{c}{\varepsilon^2}\right] \leq \frac{2^s}{\varepsilon^2 d} + \frac{d2^{-s+1}}{c\varepsilon^{-2}}$$

BJKST



Просто подбираем константы удобные, так как много степеней свободы

$$s \in [1, \dots, w]$$

$$\frac{1}{24} \le \frac{2^s}{\varepsilon^2 d} \le \frac{1}{12}$$

$$P\left[\left|\tilde{d} - d\right| \ge \varepsilon\delta\right] \le \frac{1}{12} + \frac{48}{c} \le \frac{1}{6}, \text{тогда } c = 48 \cdot 12$$

$$\text{Нижняя граница } \varepsilon^2 d < 12 \ \rightarrow d < \frac{12}{\varepsilon^2} - \text{все норм}$$

$$\text{Верхняя граница } \varepsilon^2 d > 24 \cdot 2^w \rightarrow d > \frac{2^w 24}{\varepsilon^2} - \text{уникальных ключей больше чем ключей}$$

$$P\left[\left|d - \tilde{d}\right| \ge \varepsilon\delta\right] \le \delta$$

$$P\left[\left|d - \tilde{d}\right| \ge \varepsilon\delta\right] \le \frac{1}{12} \rightarrow med\left(\tilde{d}_1 \dots \tilde{d}_m\right) =: \tilde{d}$$

$$P\left[\left|d - \tilde{d}\right| \ge \varepsilon\delta\right]$$

Неравенство Чернова

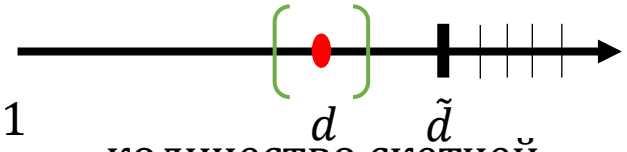
$$P[\alpha = 1] = p < \frac{1}{2}$$

$$\alpha_1 + \dots + \alpha_n = \beta$$

$$E\beta = np < \frac{n}{2}$$

$$P\left[\beta \ge \frac{n}{2}\right] \le \exp(-n)$$

Половина значений за медианой



$\frac{1}{\varepsilon^2}$ – количество скетчей,

по неравенству Чернова $\log\frac{1}{\delta} - 1$ скетч

$$O\left(\frac{1}{\varepsilon^2}\log\frac{1}{\delta}\right)$$